

SureFlow GUI Editor

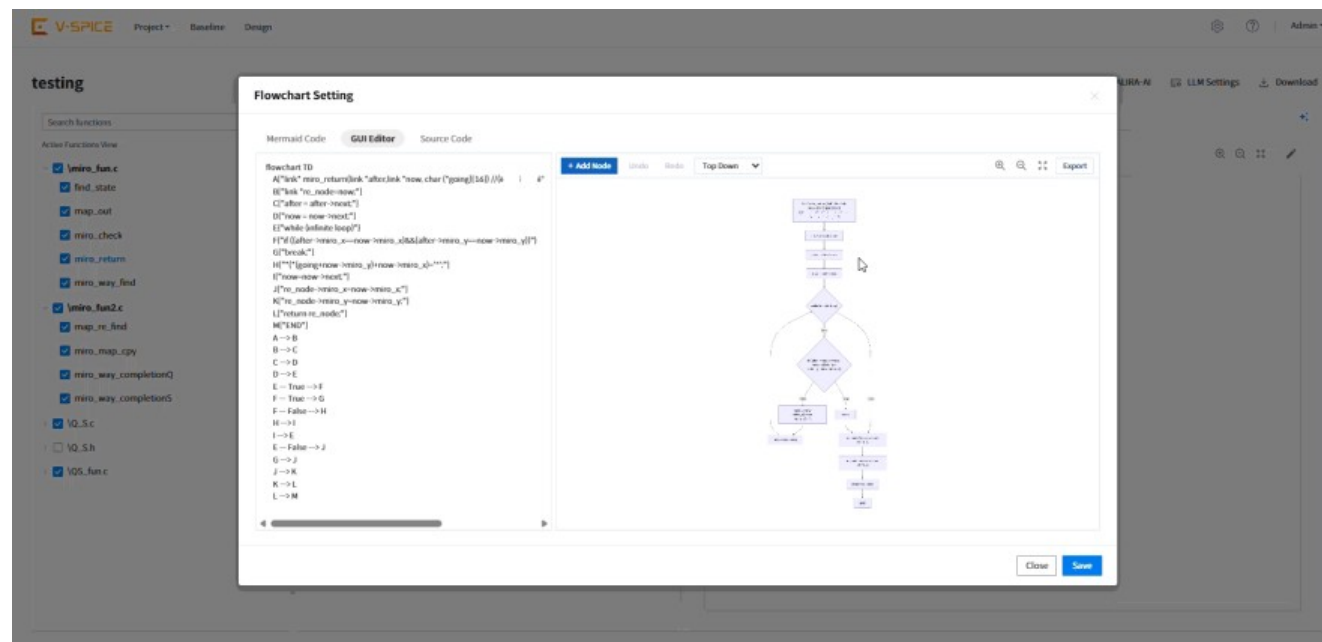
개발 및 임베드

02 SureFlow - GUI Editor 개발

SURESOFT

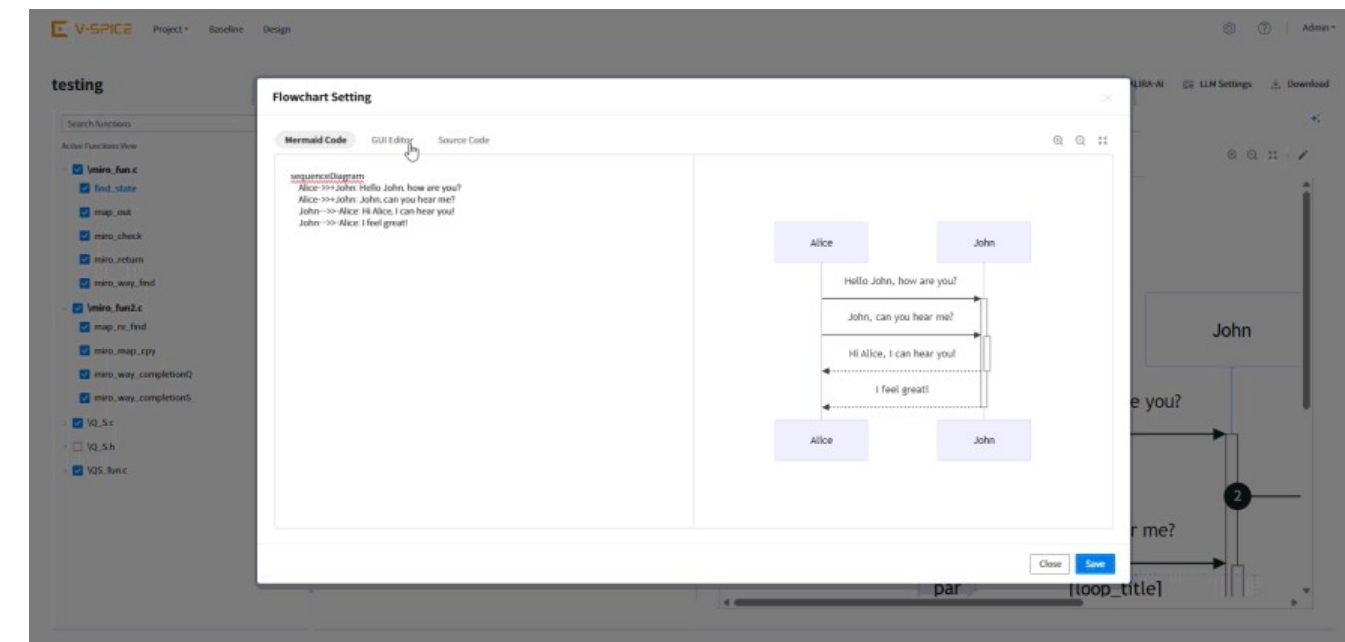
프로젝트 현재 진행상황

Flow Chart



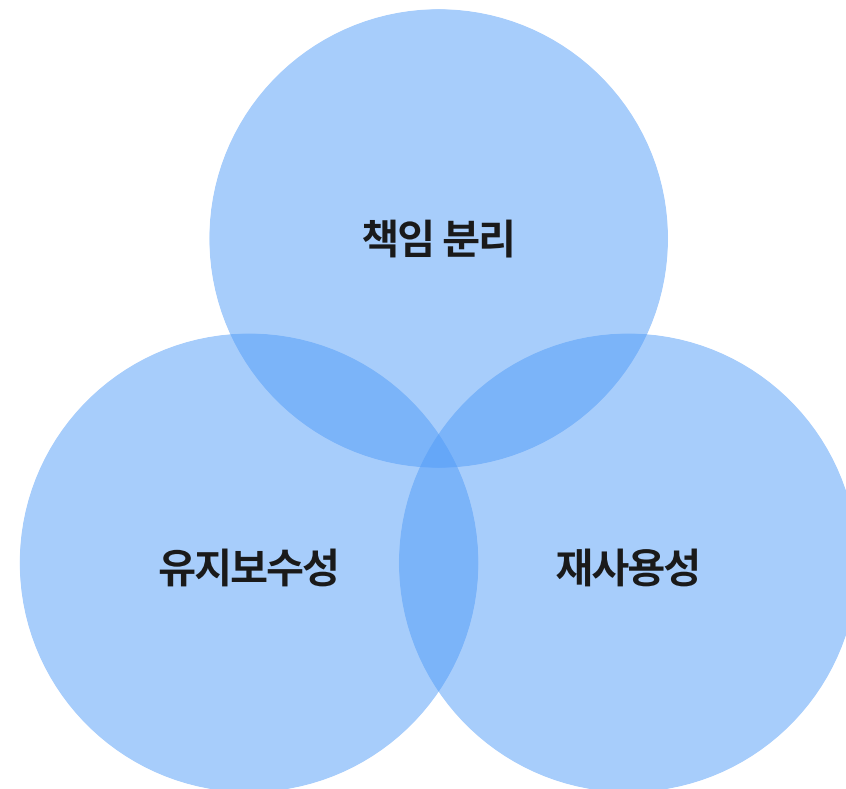
- FlowChart / Sequence Diagram GUI 편집 기능 고도화
- Mermaid 문법 기반 파싱·생성 로직 구현 및 안정화
- 쉽게 삽입 가능한 임베드 구조를 설계하고 공통 모듈 형태로 정리

Sequence diagram



- 실제 V-SPICE 환경에 임베드 적용 후 동작 테스트 및 안정화 진행
- 편집 편의성 향상을 위한 유틸 및 보조 기능 추가 구현
- 유지보수성을 높이기 위해 구조 개선 및 리팩토링 진행

프로젝트 현재 진행상황



발표 흐름

1. 아키텍처 및 구동 원리
2. 기능 고도화 내용
3. 버그 해결 및 안정화
4. 임베드 방식 및 외부 연동 방식

02 SureFlow - GUI Editor 개발

SURESOFT

아키텍처 및 구동 원리

핵심 작동 원리

svg 렌더링은 mermaid.js 엔진 기반 !

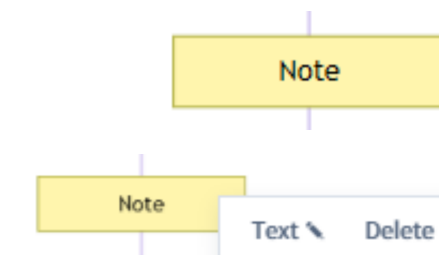


핵심 상태 정리

- script
 - 사용자가 최종적으로 저장하거나 외부 시스템에 전달하는 Mermaid 문자열
- ★ model
 - GUI 편집을 위해 문자열을 객체로 구조화한 내부 데이터
- svg
 - Mermaid가 실제로 렌더링한 결과물
- interaction
 - SVG 위에서 발생하는 클릭, 드래그, 더블클릭, 인라인 편집 같은 사용자 조작

note over Alice: Note

```
{ type: 'note', participants: ['Alice'], text: 'Note' }
```



아키텍처 및 구동 원리

model이 필요한 이유

GUI는 문자열을 직접 편집하기보다, 구조화된 객체를 편집하는 게 훨씬 쉽고 안전하기 때문!

model은 GUI가 다루기 좋은 내부 중간 데이터.

```
{
  type: 'flowchart',
  direction: 'TD',
  nodes: [
    { id: 'A', text: 'Start', shape: 'rect' }
  ],
  edges: [
    { from: 'A', to: 'B', text: '', type: '-->' }
  ]
}
```

이 구조가 있으면 GUI는 문자열 대신 그냥 객체를 수정하면 됨.

- 노드 추가 = nodes.push(...)
- 엣지 추가 = edges.push(...)
- 텍스트 수정 = 해당 객체 필드만 변경
- message 방향 반전 = from, to swap

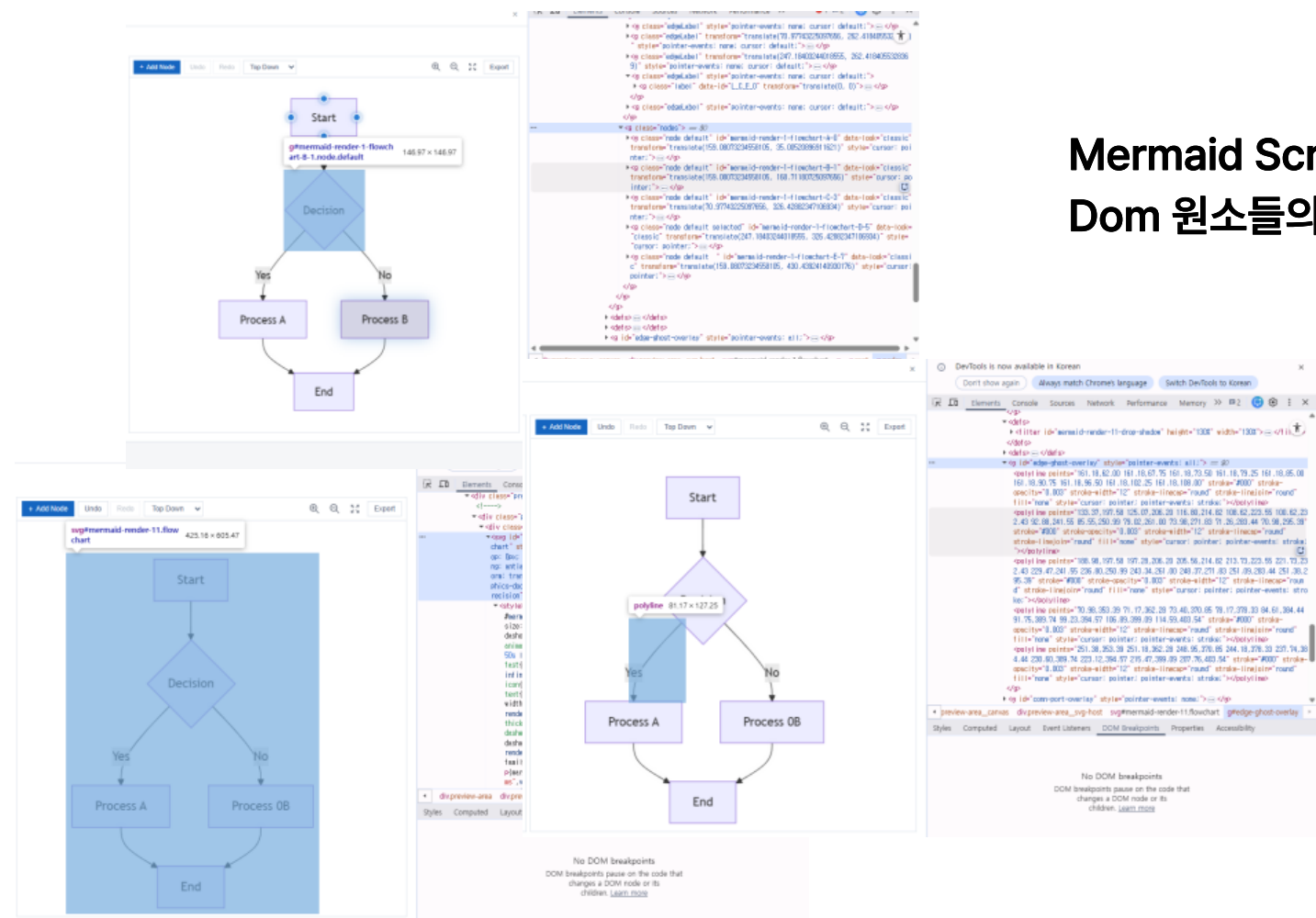
즉 GUI 액션을 객체 조작으로 쉽게 바꿔주는 GUI 편집 단위 = model.

02 SureFlow - GUI Editor 개발

SURESOFT

아키텍처 및 구동 원리

model이 필요한 이유 2. - (Feat. How to action handler attach to svg)



Mermaid Script로 렌더링 된 Flowchart Svg는 Dom 원소들의 묶음으로 구성되어있음.

SVG가 하나의 의미 있는 편집 객체로 있는 게 아니라, 브라우저 안에서는 여러 개의 태그 조각으로 분해된 상태이다.

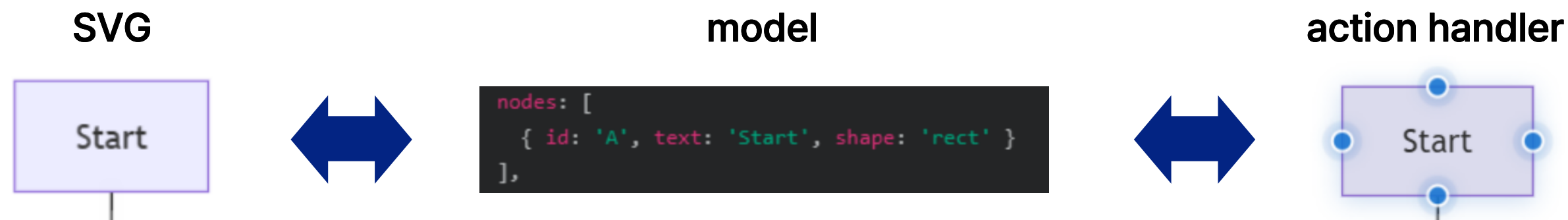
이를 편집 가능한 의미 단위로 다루려면 model 같은 중간 구조가 필요!

02 SureFlow - GUI Editor 개발

SURESOFT

아키텍처 및 구동 원리

model이 필요한 이유 2 (Feat. How to action handler attach to svg)



Script to SVG Render 후
SVG DOM 요소들을 수집해서 model과 매핑

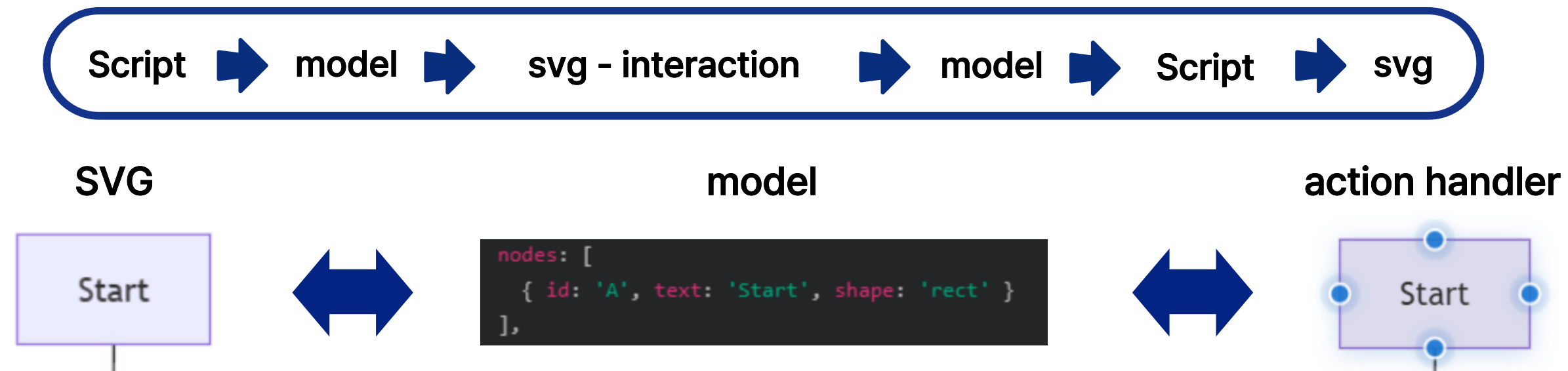
" 이 SVG 요소가 model의 몇 번째 node/edge인지 연결 "

이 매핑 결과를 바탕으로 해당 SVG DOM에
손쉽게 action handler를 붙일 수 있음

" 편집 발생 시에도 연결된 model 기반으로 재계산 쉽게 가능 "

아키텍처 및 구동 원리

계층 분리의 필요성



이런 구조에서 기능을 한 컴포넌트에 다 때려넣으면 유지보수 급격히 어려워짐.
그래서 역할별로 계층을 분리해서, 각 레이어가 한 가지 책임만 가지도록 설계를 리팩토링했습니다.

02 SureFlow - GUI Editor 개발

SURESOFT

아키텍처 및 구동 원리

계층 분리

상위 계층이 하위 계층을 사용하고, 하위 계층은 상위 계층을 몰라야 한다.

→ 의존성 순환과 변경 영향 범위를 최소화해 재사용성과 유지보수성 확보

전체적인 흐름

- UI Interaction
- Container (요청 접수)
- Model Editing → nextModel 반환
- Container가 model 교체 (결과 반영)
- Representation으로 script 재생성
- Preview Runtime 재렌더
- SVG Mapping
- Interaction 재부착

상위

하위

Container Layer

- [MermaidFullEditor.js](#)
 - editor 전체 상태 소유 (파이프라인의 중간 허브)
 - 하위 UI와 editing layer 연결

Representation Layer

- [MermaidParser / MermaidGenerator.js](#)
 - script ↔ model 변환
 - Mermaid 표현 규칙 해석
 - model 직렬화/역직렬화

Model Editing Layer

- [flowchartModelEditing.js](#)
 - model 수정만 담당
 - add / update / delete / reverse / wrap
 - 입력: model + payload ⇒ 출력: nextModel

Preview Layer

- [MermaidPreview.js](#)
 - model을 SVG로 렌더
 - 렌더 후 인터랙션 연결 준비

SVG Mapping Layer

- [SvgPositionTracker / PreviewCtxBuilder](#)
 - 렌더된 SVG DOM을 다시 읽어 model과 매핑
 - handler용 ctx bridge 제공

Interaction Layer

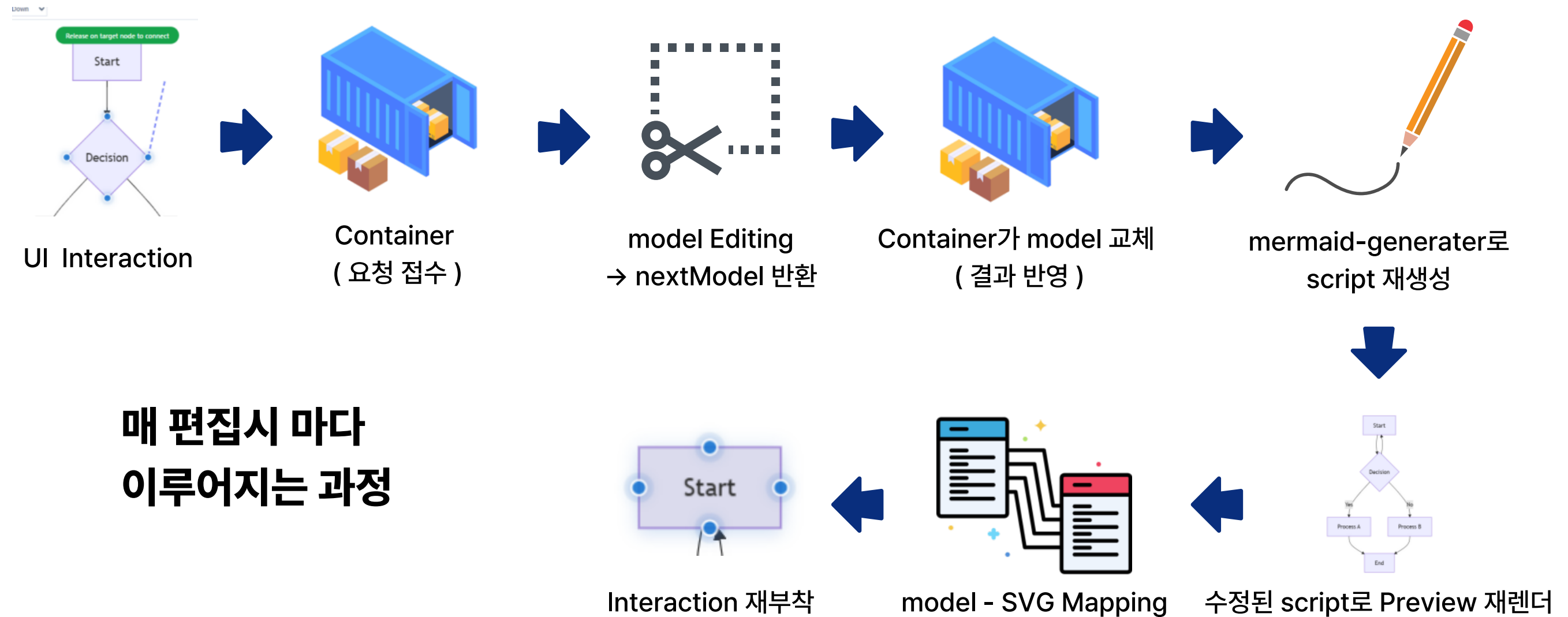
- [SvgNodeHandler / PortDragHandler](#)
 - SVG DOM에 이벤트 부착
 - 클릭/드래그를 model 대상 id/index로 계산
 - 상위에 action 요청 전달

02 SureFlow - GUI Editor 개발

SURESOFT

아키텍처 및 구동 원리

계층 분리 - GUI 편집 작동 흐름



아키텍처 및 구동 원리

계층 분리 - 통신 방식 (Feat. emit)

현재 Sure Flow 작동 흐름

사용자는 MermaidFullEditor를 직접 건드리는 게 아님!
화면에 렌더된 SVG DOM 위를 클릭/드래그



그 이벤트를 제일 먼저 받는 건
SvgNodeHandler, PortDragHandler 같은
하위 interaction 계층

하지만 진짜 상태는 "상위 Container 계층"에서 가지고 있음.

그래서 하위 계층에서 상위 계층으로, 상태 변경을 알릴 수 있는 "통신 방식" 필요

그렇다고 상위에서 하위를 직접 참조로 호출해버리면,
하위가 상위를 알아야하기 때문에 재사용성이 깨지고 계층이 뒤집혀 구조가 꼬여버림...

02 SureFlow - GUI Editor 개발

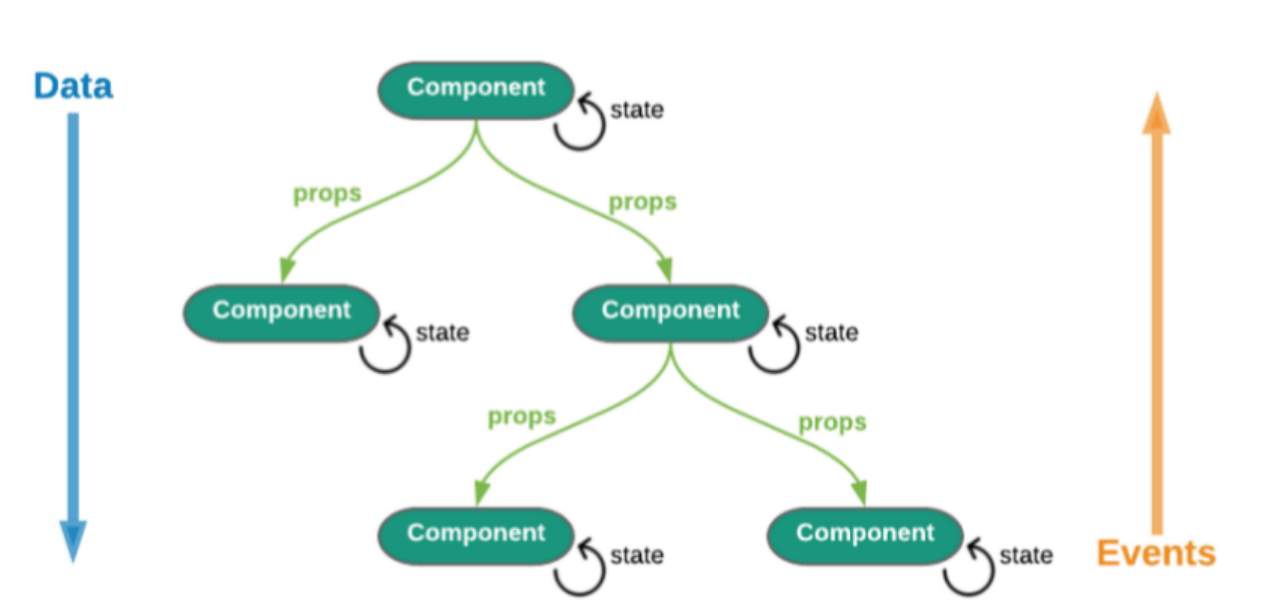
SURESOFT

아키텍처 및 구동 원리

계층 분리 - 통신 방식 (Feat. emit)

그래서 **.emit(...)** 등장

Vue에서 제공하는 이벤트 통신 방식



“데이터는 props로 내려가고, 이벤트는 emit으로 올라간다”

```
const emitClick = () => {  
  ctx.emit("emitChange", "emit");  
};  
  
return { click, emitClick };  
},
```

자식은 이벤트 발생
부모는 이벤트 리스닝
이벤트 기반 통신 기법 !

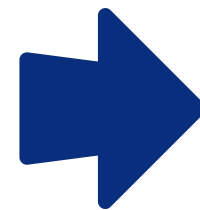
아키텍처 및 구동 원리

계층 분리 - 통신 방식 (Feat. emit)

Interaction Layer - Action Handler

GUI 편집 인터랙션을 실제 model에 적용되게 계산을 해야함.
이 과정에서 여러 복합적인 기능들이 작용.

- 상위에 이벤트 올리기
- 현재 model 일부 조회하기
- 입력창 열기
- preview rect 구하기
- 선택 강조 watcher 붙이기



- Action handler가 MermaidPreview 전체를 직접 알게 하면 결합도가 너무 커짐.
- 그렇다고 Action Handler에 필요 기능을 직접 붙이면 너무 무거워지고 중복 多

handler는 혼자 다 처리할 수 없고, 상위 기능 일부를 빌려 써야함.

아키텍처 및 구동 원리

계층 분리 - 통신 방식 (Feat. emit)

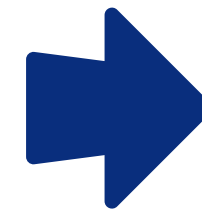
CTX = 일종의 Bridge 객체 !!

그래서 CTX(context 객체) 도입

부모가 허용한 기능 묶음만 ctx로 받아서 쓰는 제한된 인터페이스

- preview 상태 변경: setState(patch)
- 상태 조회: getState(), getModel()
- model 탐색: findNode(),
- findSequenceMessage()
- UI 보조: focusEditInput(),
- openSequenceBlockEdit()
- 렌더 보조: getPreviewRect(),
- panPreviewBy()

자식은 ctx로 내려받은
부모의 기능들을
일종의 API처럼 사용 가능 !



문제점

- SvgNodeHandler용 ctx
- SvgEdgeHandler용 ctx
- SequenceSvgHandler용 ctx

각 handler마다 다른 모양의 ctx를 직접 만들어 넘겨야 함.
ctx API가 바뀔 때 수정 지점이 흩어짐.

02 SureFlow - GUI Editor 개발

SURESOFT

아키텍처 및 구동 원리

계층 분리 - 통신 방식 (Feat. emit)

그래서 **CTX Builder 도입** (PreviewCtxBuilder.js)

ctx를 일관되게 만드는 팩토리이자, 하위 계층이 쓰는 공통 인터페이스를 한 군데서 관리하는 곳

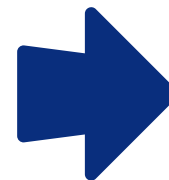
SvgNodeHandler

ctx.setState, ctx.getState, ctx.getPreviewRect,
ctx.emit, ctx.watchSelection, ctx.findNode,
ctx.focusEditInput

SvgEdgeHandler

ctx.watchEdgeSelection, ctx.getState,
ctx.getPreviewRect, ctx.setState, ctx.emit,
ctx.getModel

Action handler들이 서로 비슷한 상위 기능을 많이 씀



Lite CTX

용도: toolbar / 편집 confirm용

내용: 공통 브리지 API만 있음

emit, getState, setState, getModel 등등...

Full CTX

용도: SVG 렌더 직후 handler attach용

내용: lite ctx + 렌더 후 인터랙션에 필요한 기능

watchSelection, watchEdgeSelection 등등..

용도에 맞게 두가지 타입의 CTX Builder 구현 ⇒ 한 곳에서 공통 관리 가능!

재사용성, 유지보수성 확보!



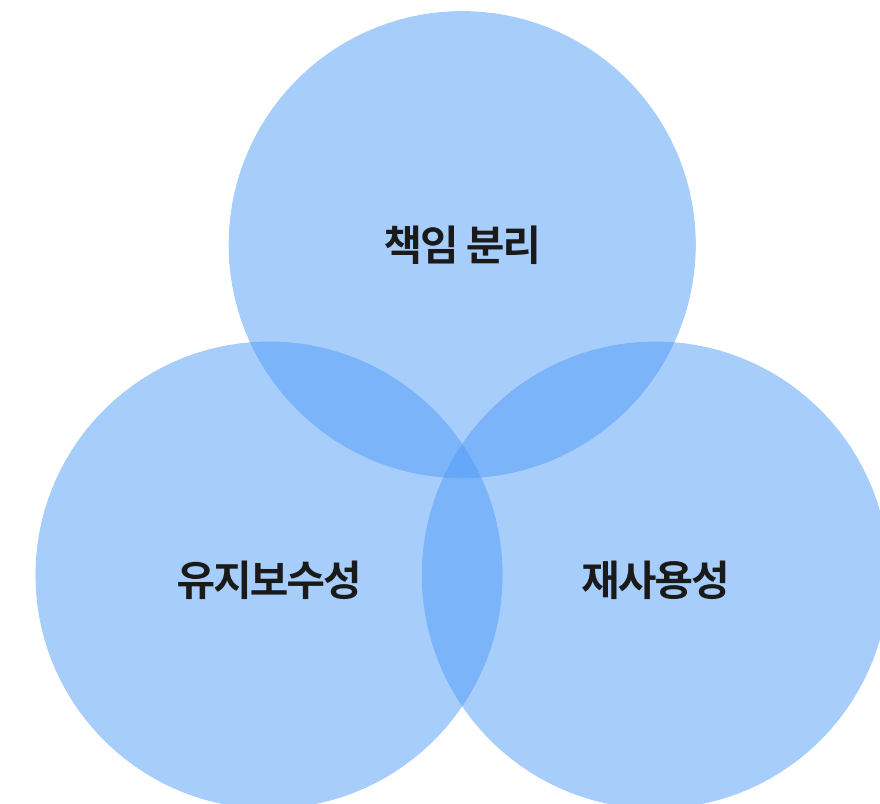
02 SureFlow - GUI Editor 개발

SURESOFT

아키텍처 및 구동 원리

폴더 구조

```
gui-editor/  
├─ src/  
│   ├── components/      # 에디터, 툴바, 프리뷰 컴포넌트  
│   │   └─ mixins/       # 공통 액션/토스트/export 믹스인  
│   ├── actions/         # SVG 상호작용 처리 핸들러  
│   ├── representation/  # Mermaid script <-> model 변환  
│   ├── model-editing/   # flowchart/sequence 모델 수정 로직  
│   └─ utils/            # codec, history, export, diagnostics, preview ctx  
├─ dist/                 # 번들 결과물  
├─ docs/                 # 설계/히스토리/임베드 문서  
├─ build.js              # 번들 빌드 스크립트  
├─ index.html            # 로컬 데모 페이지  
├─ GuiEditor.css         # 컴포넌트 스타일  
├─ style.css             # 데모 페이지 스타일  
└─ README.md            # 프로젝트 개요
```



02 SureFlow - GUI Editor 개발

SURESOFT

아키텍처 및 구동 원리

폴더 구조 - 세부

Container Layer & Preview Layer

```
gui-editor/  
├─ src/  
│  ├─ app.js                # 로컬 데모/부트스트랩  
│  ├─ components/  
│  │  ├─ MermaidEditor.js   # 텍스트 에디터  
│  │  ├─ MermaidFullEditor.js # 외부 공개용 메인 컴포넌트  
│  │  ├─ MermaidLiveEditor.js # 라이브 편집 UI  
│  │  ├─ MermaidPreview.js  # Mermaid SVG 렌더/프리뷰  
│  │  └─ MermaidToolbar.js  # 편집 툴바  
│  │  └─ mixins/  
│  │     ├─ exportMixin.js   # export 관련 공통 로직  
│  │     ├─ flowchartActionsMixin.js # flowchart 액션 wrapper  
│  │     └─ sequenceActionsMixin.js # sequence 액션 wrapper  
│  └─ toastMixin.js         # toast 표시 로직
```

- UI 구성 요소는 components
- 공통 편집 동작은 mixins

Representation Layer

```
└─ representation/  
  ├─ mermaid-generator.js    # flowchart model -> script  
  ├─ mermaid-parser.js       # script -> flowchart model  
  ├─ sequence-generator.js    # sequence model -> script  
  └─ sequence-parser.js       # script -> sequence model
```

- 스크립트 해석은 representation
(파싱 - 직렬화 담당)

Model-Editing Layer

```
└─ model-editing/  
  ├─ flowchartModelEditing.js # flowchart 모델 편집  
  └─ sequenceModelEditing.js  # sequence 모델 편집
```

- model 수정은 model-editing
(편집 내용 기반 model 객체 수정)

SVG Mapping Layer & Interaction Layer

```
└─ actions/  
  ├─ PortDragHandler.js      # 포트 드래그 연결  
  ├─ SequenceBlockHandler.js # sequence block 상호작용  
  ├─ SequenceMessageDragHandler.js # sequence message 드래그  
  ├─ SequencePositionTracker.js # sequence 위치 추적  
  ├─ SequenceSvgHandler.js   # sequence SVG 핸들링  
  ├─ SvgEdgeHandler.js       # flowchart edge 선택/편집  
  ├─ SvgNodeHandler.js       # flowchart node 상호작용  
  └─ SvgPositionTracker.js    # SVG 좌표 추적
```

- GUI 상호작용 로직은 actions
(SVG 인터렉션 담당)

utils

```
└─ utils/  
  ├─ FlowEdgeCodec.js        # edge 인코딩/디코딩  
  ├─ HistoryManager.js       # undo/redo 관리  
  ├─ IdAllocator.js          # ID 발급  
  ├─ ModelDiagnostics.js     # 모델 진단  
  ├─ ParserHighlight.js      # 파서 하이라이트  
  ├─ PreviewCtxBuilder.js    # preview/action 공통 ctx 생성  
  ├─ SequenceMessageCodec.js # sequence message codec  
  ├─ SequenceStatementUtils.js # sequence statement 유틸  
  ├─ StorageManager.js       # 저장 관련 유틸  
  └─ SvgExport.js            # SVG/PNG export
```

- 공통 유틸 기능은 utils
(Export 기능, History 기능 등등)

docs & dist

```
└─ docs/                      # 설계/리팩터링/임베드 문서들  
└─ dist/  
  ├─ gui-editor.component.js  # 빌드된 컴포넌트 번들  
  └─ GuiEditor.css            # 배포용 스타일  
└─ build.js  
└─ index.html  
└─ GuiEditor.css  
└─ style.css  
└─ README.md  
└─ LICENSE
```

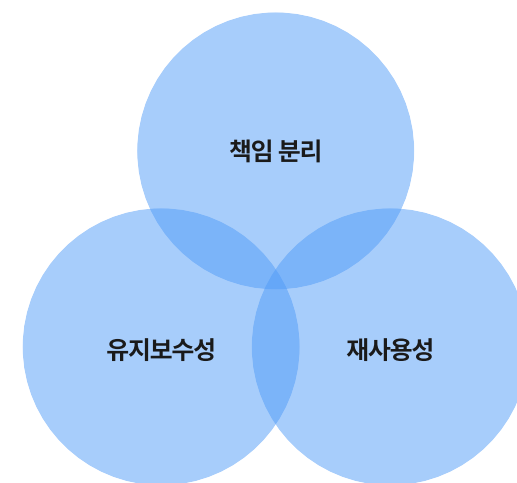
- 배포 결과물은 dist
- 설계 기록은 docs

02 SureFlow - GUI Editor 개발

SURESOFT

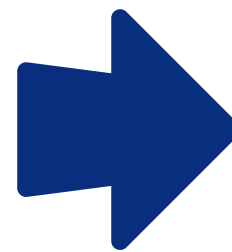
아키텍처 및 구동 원리

아키텍처 분리 이점 - 실제 예시

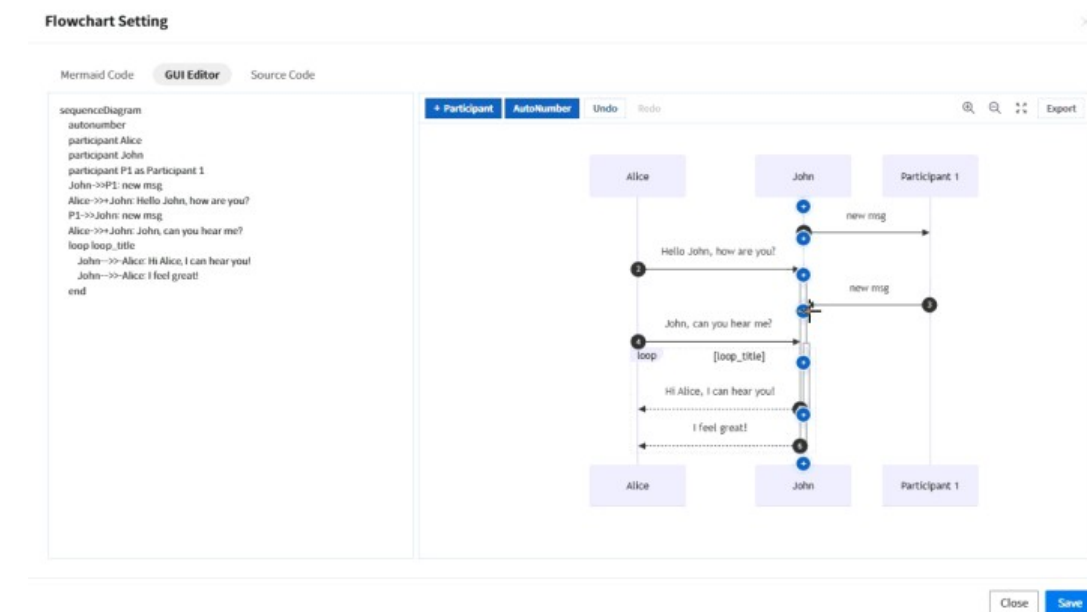


실제로 기능 하나를 추가할 때

- 어디를 수정해야 하는지 빨리 보이고
- 수정 범위가 예측 가능하고
- 다른 계층을 덜 건드리고
- 버그가 났을 때 원인 범위를 빨리 좁힐 수 있음



실제 예시



note over 기능 추가를 예시로 본
재사용성과 책임 분리 효과

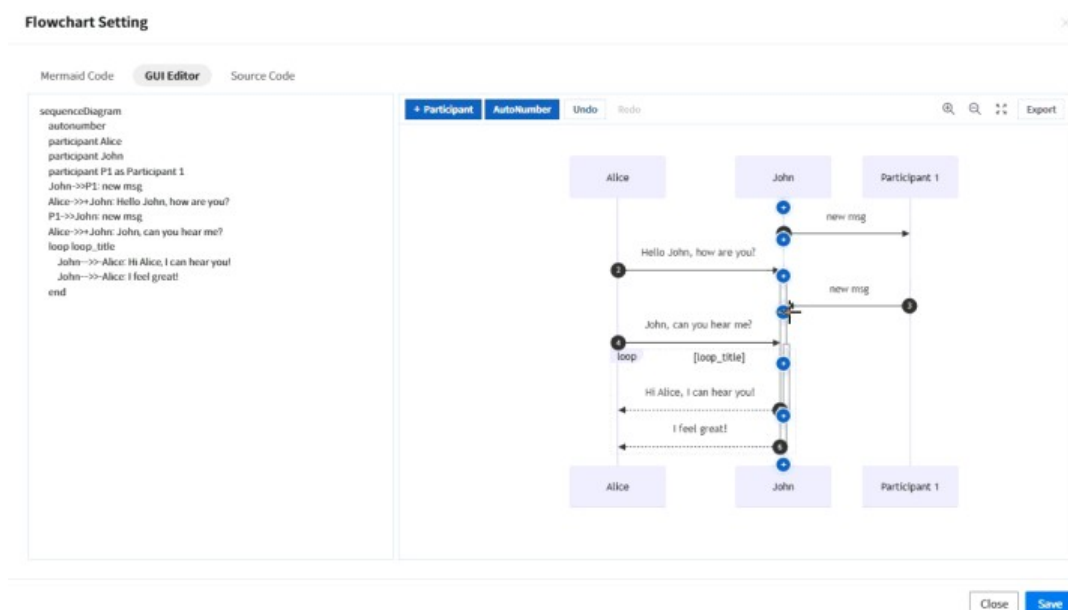
02 SureFlow - GUI Editor 개발

SURESOFT

아키텍처 및 구동 원리

아키텍처 분리 이점 - 실제 예시

실제 예시



note over 기능 추가를 예시로 본
재사용성과 책임 분리 효과

시퀀스 다이어그램에 메모 붙이는 단순한 기능 추가

1. script를 읽는 parser 기능
2. model을 다시 script로 만드는 generator 기능
3. preview에서 note를 찾고 클릭하는 interaction 기능
4. 편집 UI 상태를 여닫는 기능
5. 실제 model 변경을 수행하는 action 기능

기능은 단순해 보여도 실제로는 이런 여러 Layer들을 건드림

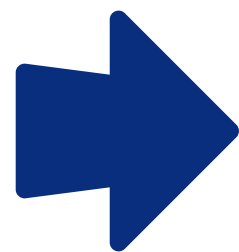
02 SureFlow - GUI Editor 개발

SURESOFT

아키텍처 및 구동 원리

아키텍처 분리 이점 - 실제 예시

- `src/representation/sequence-parser.js`
 - Mermaid script를 읽어 sequence model로 바꾸는 책임
- `src/representation/sequence-generator.js`
 - sequence model을 다시 Mermaid script로 만드는 책임
- `src/model-editing/sequenceModelEditing.js`
 - sequence 모델 편집 규칙을 담당하는 책임
- `src/utils/SequenceStatementUtils.js`
 - `statements` 배열 조작 같은 순수 유틸 책임
- `src/actions/SequenceSvgHandler.js`
 - 렌더된 SVG에서 note/message/block 같은 요소 클릭 처리 책임
- `src/utils/PreviewCtxBuilder.js`
 - action handler와 Vue component 사이를 연결하는 bridge 책임
- `src/components/MermaidPreview.js`
 - overlay input, toolbar, 편집 상태 등 화면 책임
- `src/components/mixins/sequenceActionsMixin.js`
 - editor 공통 sequence 액션을 묶는 책임



“구조가 잘 나뉘어 있으면 얼마나 적은 비용으로 확장되는가”

이미 계층을 나누어 책임을 분리해두었기에
수정해야할 위치가 바로 보이고 최소 범위로 작업 가능

기능 추가 전제: 현재 구조에서 역할이 어떻게 나뉘어 있는가

아키텍처 및 구동 원리

**note over 추가 시
실제로 수정된 책임 구간**

1 Note Over 문법 인식 추가

```
/**
 * Mermaid 시퀀스 다이어그램 파서
 * sequenceDiagram 문법을 내부 모델로 변환한다.
 */

(function (global) {
  'use strict';

  var MESSAGE_RE = SequenceMessageCodec.MESSAGE_RE;
  var BLOCK_OPEN_RE = /^(loop|alt|opt|par)(?:\{(?:\s+(.+)?)?\}$/i;
  var RAW_BLOCK_OPEN_RE = /^(rect|critical|break|box)\b(?:\{(?:\s+(.+)?)?\}$/i;
  var NOTE_OVER_RE = /*note\s+over\s+([A-Za-z0-9_\u03131-\u079D]+(?:\s+[\s*{A-Za-z0-9_\u03131-\u079D}+

  function pushBlock(model, kind, recognized) {
    model.blockStack.push({
```

`sequence-parser.js`에서 `note over ...`를 읽어
`statement.type === 'note'` 형태로 모델화

Note Over 문법 출력

```
function renderState(state, message, level) {
  if (/^(loop|alt|else|opt|par|and)$/.test(state.type)) {
  }

  if (state.type === 'note') {
    var parts = (state.participants || []).join(', ');
    return renderIndented(level || 0, 'note over ' + parts + (state.text ? ': ' + state.text : ''));
  }
}
```

sequence-generator.js에서 note statement
를 다시 **note over A: text**로 복원

2 데이터 편집

데이터 편집

```
246 addNote: function (model, data) {
247   if (!model || !data || !data.participantId) return model;
248   return finish(model, {
249     statements: SequenceStatementUtils.addNote@Statement(
250       model,
251       data.participantId,
252       data.insertIndex !== undefined ? data.insertIndex : null,
253       data.text || 'Note'
254     )
255   });
256 },
257
258 updateNoteText: function (model, data) {
259   if (!model || !data || data.statementIndex === null || data.statementIndex === undefi

function deleteBranchStatement(model) {
  var statements = cloneStatements(model);
  if (statementIndex < 0 || statementIndex > statements.length) return;
  statements.splice(statementIndex, 1);
  return statements;
}

function addNoteStatement(model, participantId, insertAtMessageIndex, text) {
  var statements = cloneStatements(model);
  var note = { type: 'note', participants: [participantId], text: text || 'Note' };
  var statementIndex = (insertAtMessageIndex !== null && insertAtMessageIndex !== undefined)
    ? messageIndexToStatementIndex(statements, insertAtMessageIndex)
    : -1;
  if (statementIndex >= 0) {
    statements.push(note);
  } else {
```

SequenceStatementUtils.js, sequenceModelEditing.js 에서 note 추가/수정/삭제 규칙 반영

3 Action Handler - Interaction

```

232+     attachNotes: function (svgEl, model, ctx) {
233+         var noteRects = Array.prototype.slice.call(svgEl.querySelectorAll('rect.note'));
234+         var noteStatements = [];
235+         var statements = (model && model.statements) || [];
236+         for (var i = 0; i < statements.length; i++) {
237+             if (statements[i] && statements[i].type === 'note') {
238+                 noteStatements.push({ statementIndex: i, statement: statements[i] });
239+             }
240+         }
241+
242+         var seenGroups = [], noteGroups = [];
243+         for (var r = 0; r < noteRects.length; r++) {
244+             var g = noteRects[r].parentNode;
245+             if (g && seenGroups.indexOf(g) === -1) { seenGroups.push(g); noteGroups.push(g); }
246+         }
247+
248+         for (var j = 0; j < Math.min(noteGroups.length, noteStatements.length); j++) {
249+             (function (noteGroup, noteInfo) {
250+                 noteGroup.style.cursor = 'pointer';
251+                 noteGroup.style.pointerEvents = 'all';

```

SequenceSvgHandler.js에서 note SVG를 찾아 클릭/더블클릭/선택 처리

5 에디터 공통 액션

```

217: @create-sequence-note~"addSequenceNote"\
218: @update-sequence-note-text~"updateSequenceNoteText"\
219: @toggle-participant-kind~"toggleParticipantKind"\
220: @move-sequence-participant~"moveSequenceParticipant"\
221: @code-selected~"toggleSelectedCode"
222:
223: addSequenceNote: function (data) {
224:   if (this.isFlowchart || !data || !data.participantId) return;
225:   this._snapshot();
226:   this._updateSequenceModel({
227:     statements: SequenceStatementUtils.addNoteStatement(
228:       this.model,
229:       data.participantId,
230:       data.insertIndex !== undefined ? data.insertIndex : null,
231:       "Note"
232:     ),
233:   });
234: },
235:
236: updateSequenceNoteText: function (data) {
237:   if (this.isFlowchart || !data || data.statementIndex === null || data.statementIndex === undefined) return;
238:   this._snapshot();
239:   this._updateSequenceModel({
240:     statements: SequenceStatementUtils.updateNoteText(this.model, data.statementIndex, data.text)
241:   });
242: },

```

sequenceActionsMixin.js, MermaidFullEditor.js, MermaidLiveEditor.js에서 생성/수정 이벤트를 받아 상태 반영

4 화면 연결

```

822+   if (this.editingSequenceNoteStatementIndex !==
823+       this.$emit('update-sequence-note-text', {
824+         statementIndex: this.editingSequenceNotes
825+         text: this.editingSequenceNoteText.trim()
826+       }));
827+   }
828+   this.editingSequenceNoteStatementIndex = null
829+   this.editingSequenceNoteText = '';
830+ },
831+
832+   cancelSequenceNoteEdit: function () {
833+     this.editingSequenceNotesStatementIndex = null
834+     this.editingSequenceNoteText = '';
835+   },
836+
837+   onSequenceNoteEditKeyDown: function (e) {
838+     if (e.key === 'Enter') { e.preventDefault(); this.confirmSequenceNoteEdit(); }
839+     if (e.key === 'Escape') { this.cancelSequenceNoteEdit(); }

```

```

openSequenceNoteEdit: function (statementIndex, text, clientX, clientY) {
877+   vm.sequenceToolBar = null;
878+   vm.editingSequenceNotesStatementIndex = statementIndex;
879+   vm.editingSequenceNoteText = text || '';
880+   vm.sequenceNoteEditStyle = {
881+     position: 'fixed',
882+     left: Math.max(12, clientX - 140) + 'px',
883+     top: Math.max(12, clientY - 22) + 'px',
884+     zIndex: 1000,
885+     width: '280px'
886+   };
887+   vm.$nextTick(function () {
888+     var el = vm.$refs.sequenceNoteInput;
889+     if (el) { el.focus(); el.select(); }
890+   });
891+ },

```

PreviewCtxBuilder.js와 MermaidPreview.js 에서 note 편집 input 열기, text 수정 이벤트 연결

아키텍처 및 구동 원리

아키텍처 분리 이점 - 실제 예시

재사용성 측면에서 얻은 효과

- **parser/generator 재사용**
 - UI와 무관하게 “문자열 ↔ 모델” 변환 로직이 독립돼 있어 그대로 재사용.
- **statement 유틸 재사용**
 - note 관련 배열 조작 규칙이 UI 코드 밖에 있어서 LiveEditor, FullEditor 양쪽에서 공통 사용.
- **CTX bridge 객체 재사용**
 - PreviewCtxBuilder.js를 통해 handler가 Vue 내부 구현을 직접 몰라도 되므로 다른 interaction에도 같은 연결 방식을 재사용.
- **action 계층 재사용**
 - note 처리도 기존 sequence SVG 처리 흐름 안에 자연스럽게 들어가며, 새 기능이 기존 상호작용 구조를 재활용.

책임 분리 측면에서 얻은 효과

- **어디를 고쳐야 할지가 명확**
 - 문법은 `representation`
 - 모델 규칙은 `model-editing` / `utils`
 - SVG 상호작용은 `actions`
 - 화면 입력/툴바는 `components`
- **변경 영향 범위가 좁아짐**
 - note 문법을 바꿔도 전체 UI를 뒤집을 필요가 없음
- **디버깅이 쉬워짐**
 - 파싱 문제인지, 모델 수정 문제인지, 프리뷰 클릭 문제인지 층별로 바로 좁힐 수 있음.

결과적으로

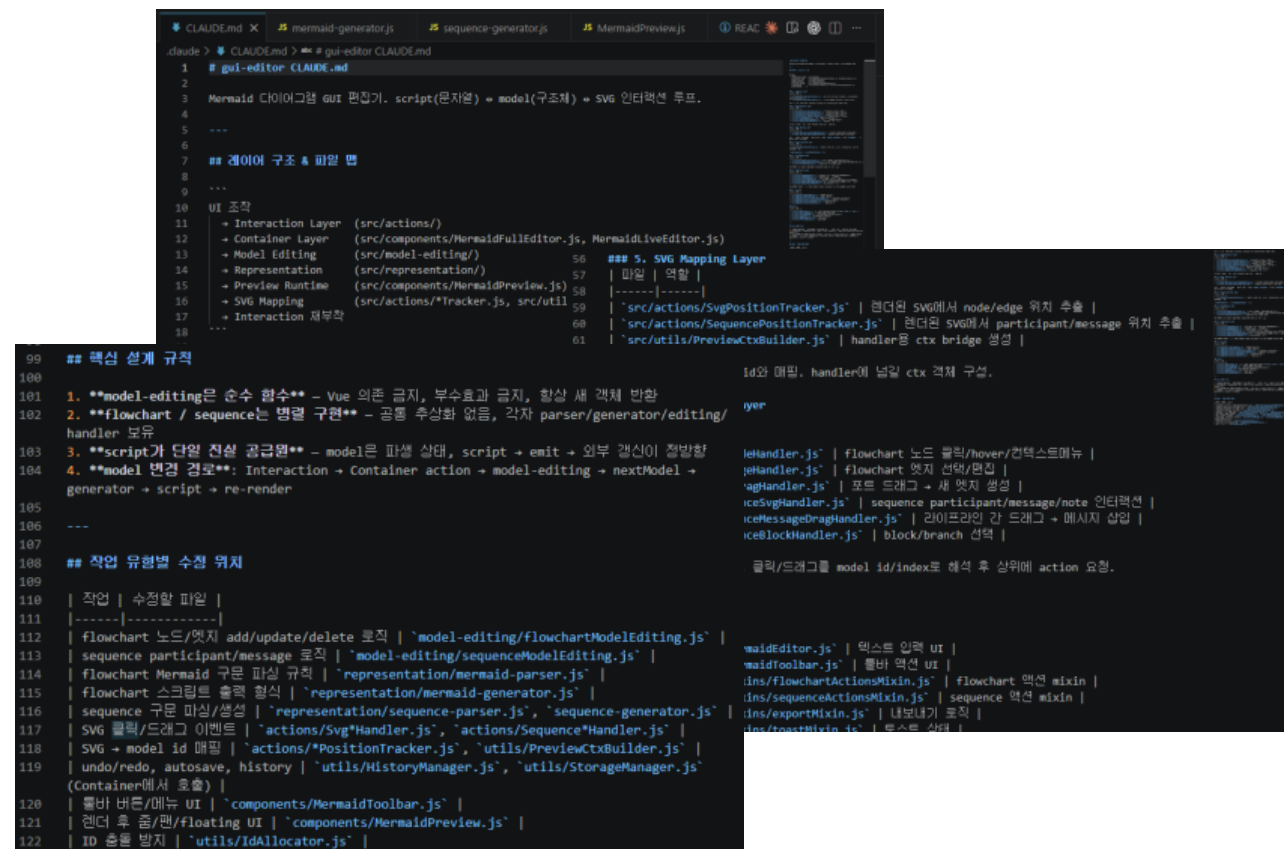
기능 추가 비용, 영향 범위, 디버깅 난이도를 모두 낮췄습니다.

02 SureFlow - GUI Editor 개발

SURESOFT

아키텍처 및 구동 원리

아키텍처 분리를 통한 AI Agent 개발 효율 개선



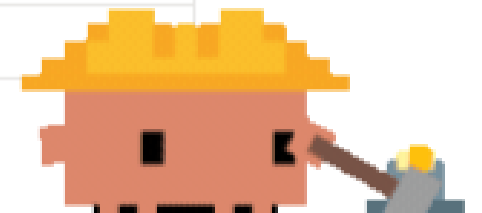
이렇게 분리한 아키텍처를 CLAUDE.md 문서로 정리

덕분에 AI Agent가 작업할 때도
전체 코드를 무작정 탐색하지 않고,
요청에 맞는 계층과 파일을 빠르게 찾을 수 있었습니다.

그 결과, 불필요한 코드 탐색이 줄어들고
토큰 사용량과 작업 시간도 함께 줄일 수 있었습니다.

	아키텍처 O + CLAUDE.md O	아키텍처 O + CLAUDE.md X	아키텍처 X + CLAUDE.md X
탐색	~7,000 tokens	~9,000 tokens	~11,000 tokens
구현	~1,500 tokens	~1,500 tokens	~2,000 tokens
합계	~8,500 tokens	~10,500 tokens	~13,000 tokens
시간	~5분	~7분	~10분

note over 기능 추가 작업 기준 - 총 토큰 35% 절감 / 시간 50% 절감



기능 고도화 내용

1. Mermaid PlayGround 보다 나은 부분
2. 기능 고도화 부분
3. 사용성 고도화 부분

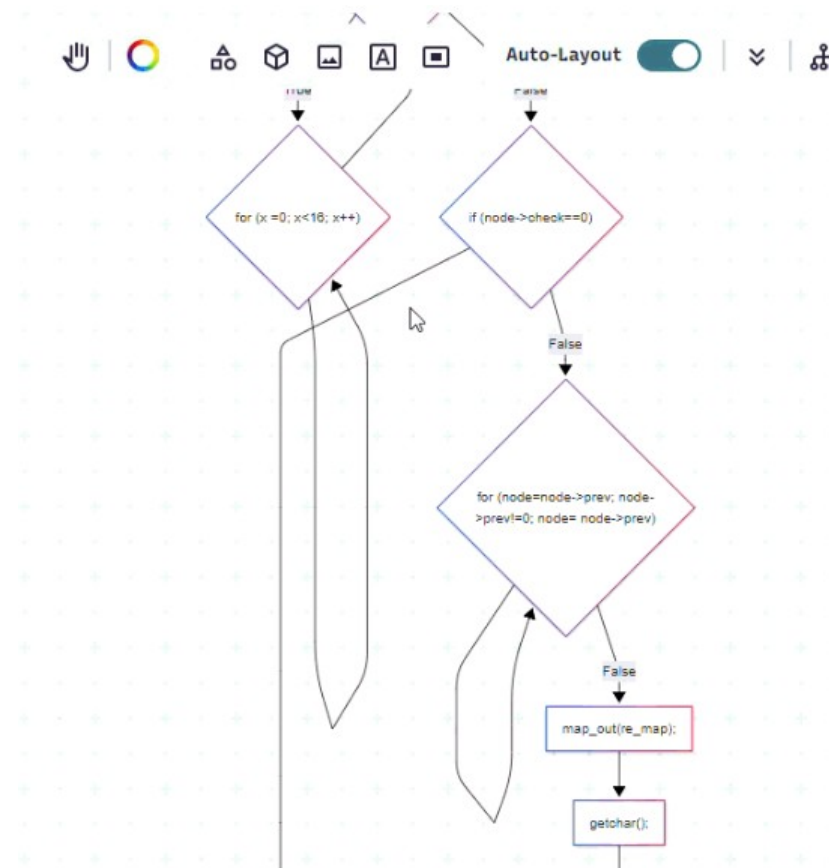
02 SureFlow - GUI Editor 개발

SURESOFT

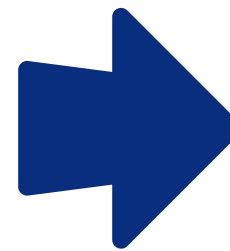
기능 고도화 내용

1. Mermaid PlayGround 보다 나은 부분

Self-Loop Edge 버그 해결

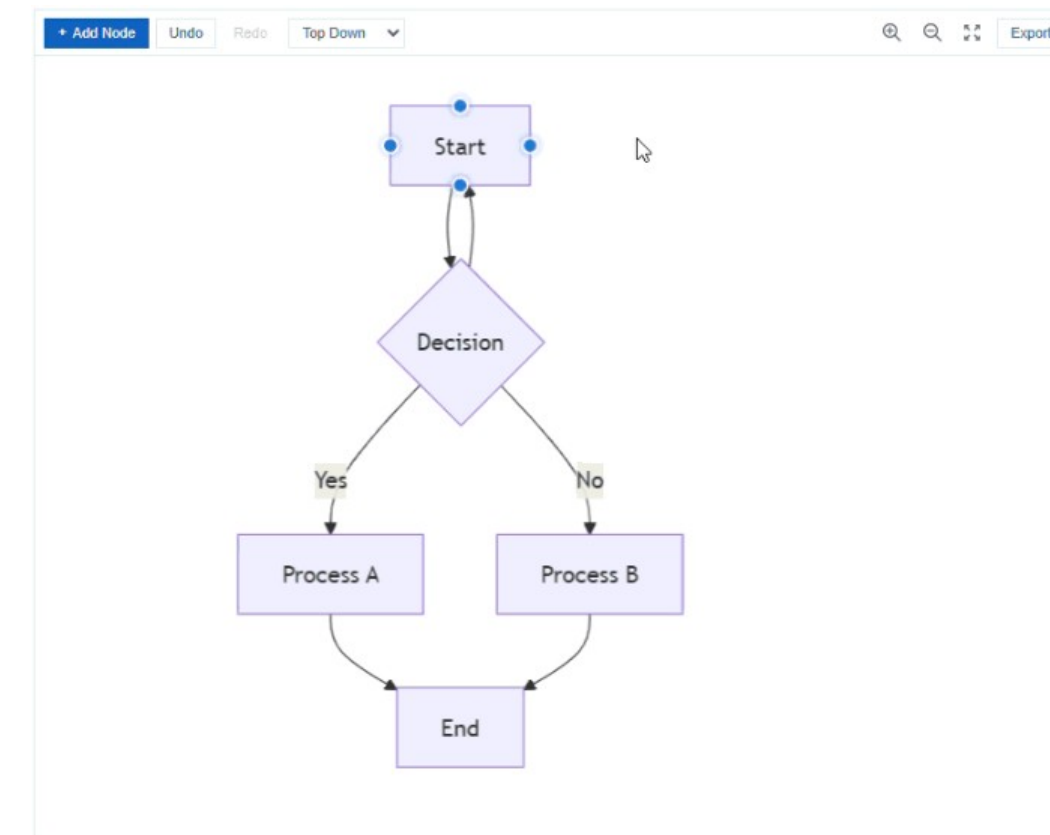


Mermaid PlayGround 내 Self Loop Edge 편집 버그



self loop는 동일 노드에서 출발·회전·복귀하는 구조라
Mermaid SVG에서 3개의 path 조각으로 분해 렌더되는 원인

self loop를 3개 렌더 슬롯으로 간주하는 보정 로직을
추가해 동일 model edge로 다시 매핑함으로써 해결



Self Loop Edge 편집 버그 해결 및
GUI 기반 Self Loop Edge 추가 기능 구현

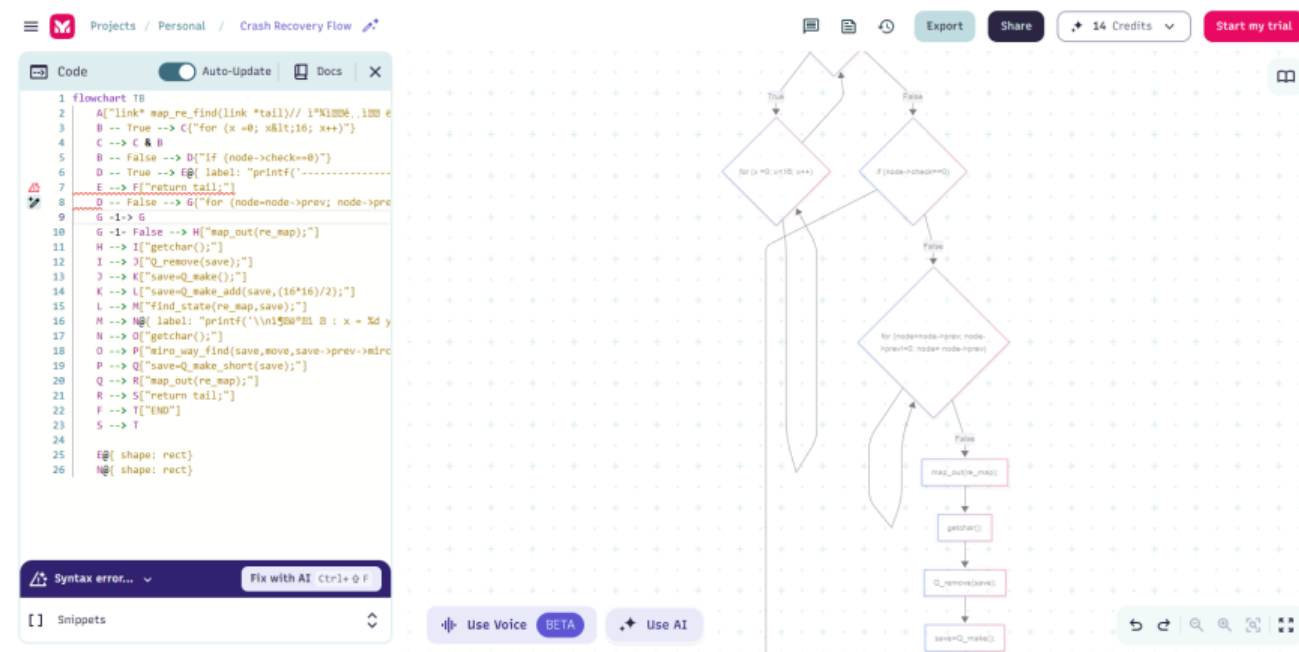
02 SureFlow - GUI Editor 개발

SURESOFT

기능 고도화 내용

1. Mermaid PlayGround 보다 나은 부분

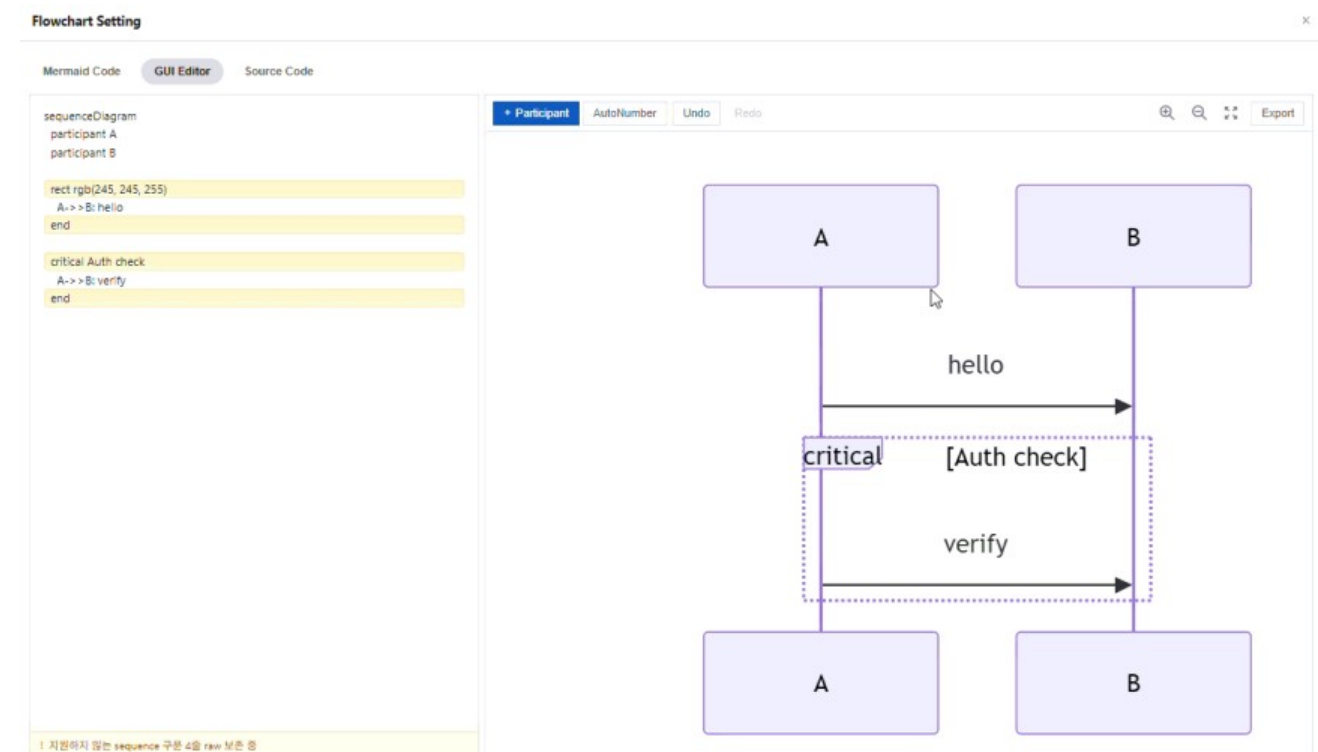
미지원 문법 및 파싱 에러질 보존 및 하이라이트 기능



Mermaid Playground는 어디가 문제인지 애매하게 뜨고 파싱 안된 줄이 있을 시 전체 수정이 안되게 잠김.

파서에서 미지원 문법이나 파싱 실패 줄을 삭제하지 않고 보존해 원문과 위치 정보를 남김. ⇒ 보존된 체로 수정 가능

그다음 generator는 원문을 그대로 다시 출력하고, 에디터는 하이라이트해서 미지원 구간을 사용자에게 표시.



미지원 문법이거나 문법이 깨진 줄은 보존 및 하이라이트 표시 사용자가 어디가 문제인지 바로 알 수 있는 UX 기능

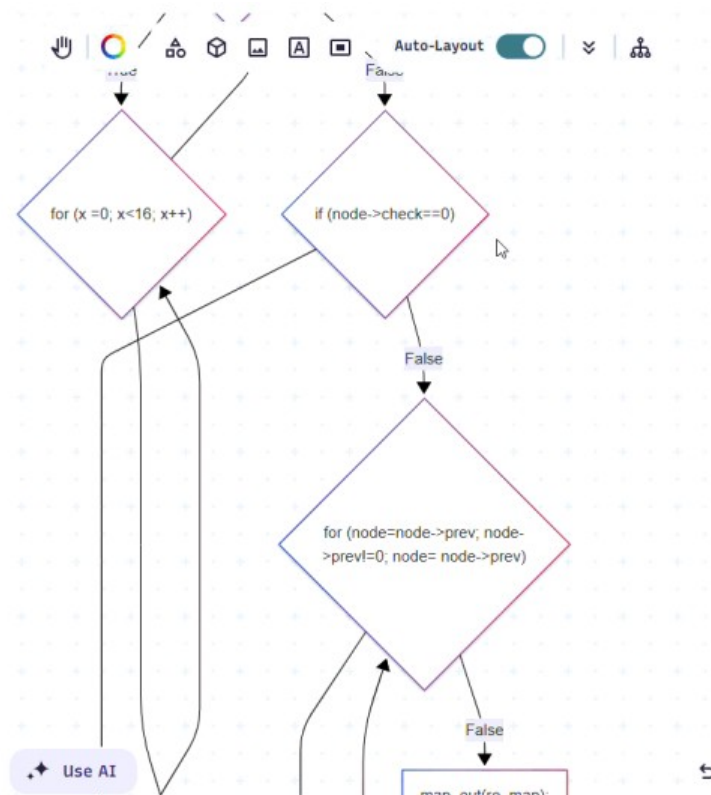
02 SureFlow - GUI Editor 개발

SURESOFT

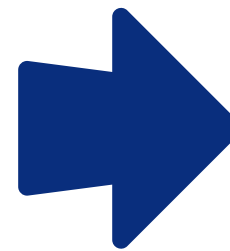
기능 고도화 내용

1. Mermaid PlayGround 보다 나은 부분

포트 드래그 상황 - 오토팬 기능 추가

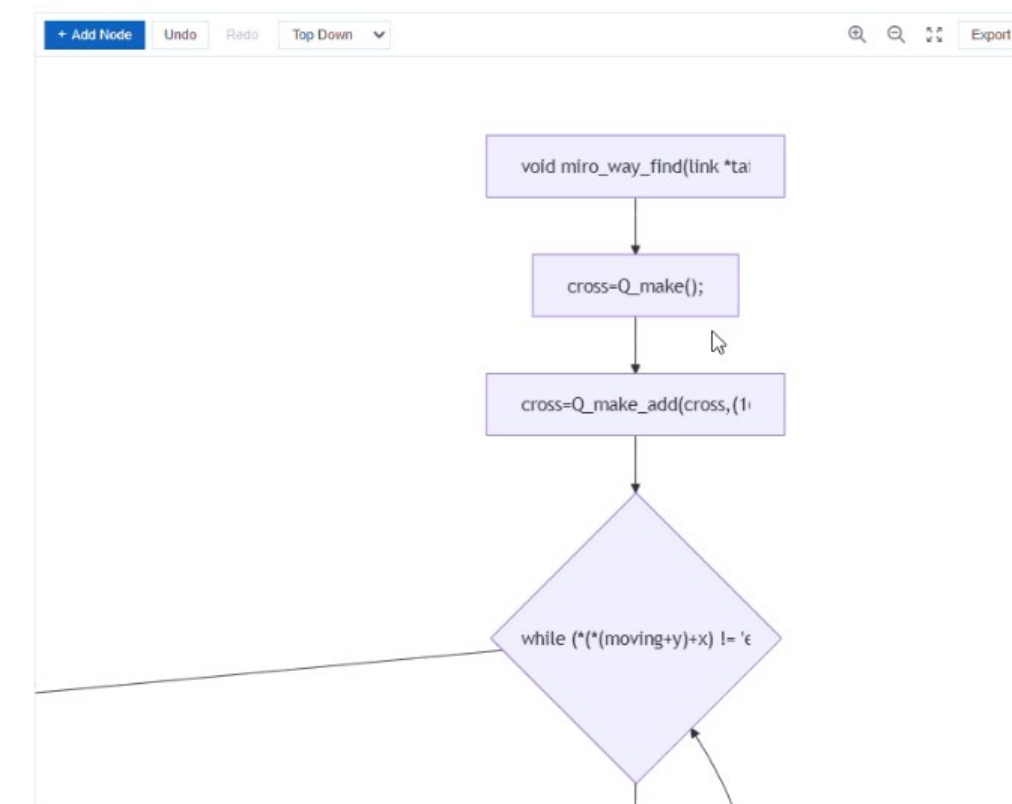


Mermaid PlayGround는 엣지 연결 시 시점 이동(오토팬) 기능이 없어 현 시야 외에 노드 연결이 불가능해서 불편함.



포트 드래그 중 커서가 프리뷰 가장자리로 가면 현재 마우스 위치를 기준으로 가장자리 근접 여부를 계산해 캔버스를 자동으로 스크롤하도록 로직 구축

그래서 큰 다이어그램에서도 드래그를 끊지 않고 화면 밖 노드까지 자연스럽게 엣지를 연결할 수 있게 개선



오토팬 기능을 추가해서 규모가 큰 다이어그램에서도 편리하게 편집 가능하게 UX 개선

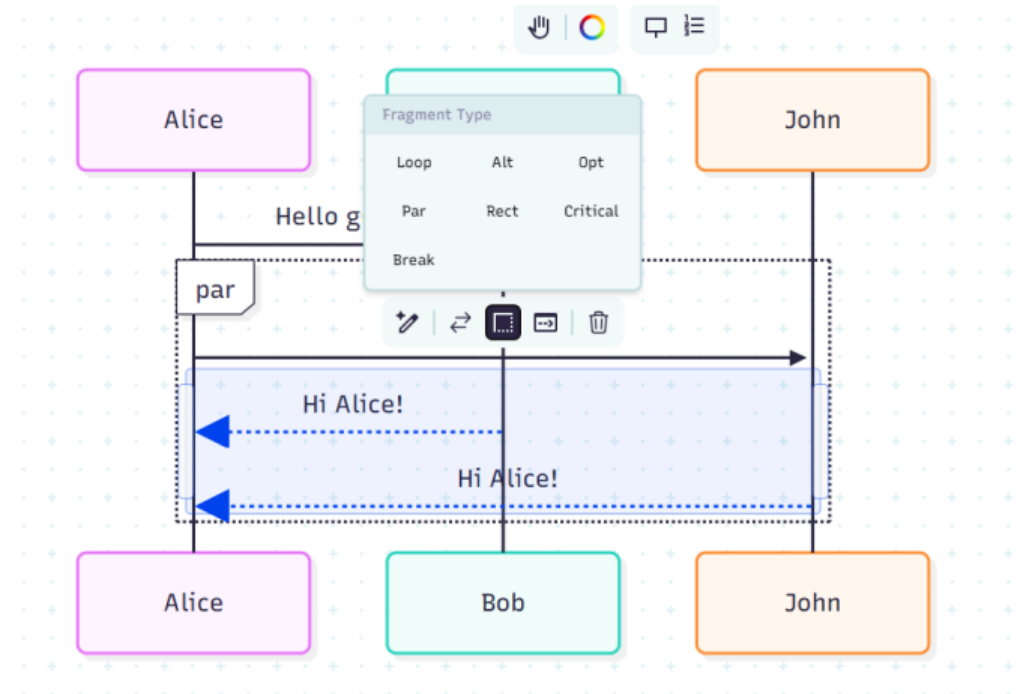
02 SureFlow - GUI Editor 개발

SURESOFT

기능 고도화 내용

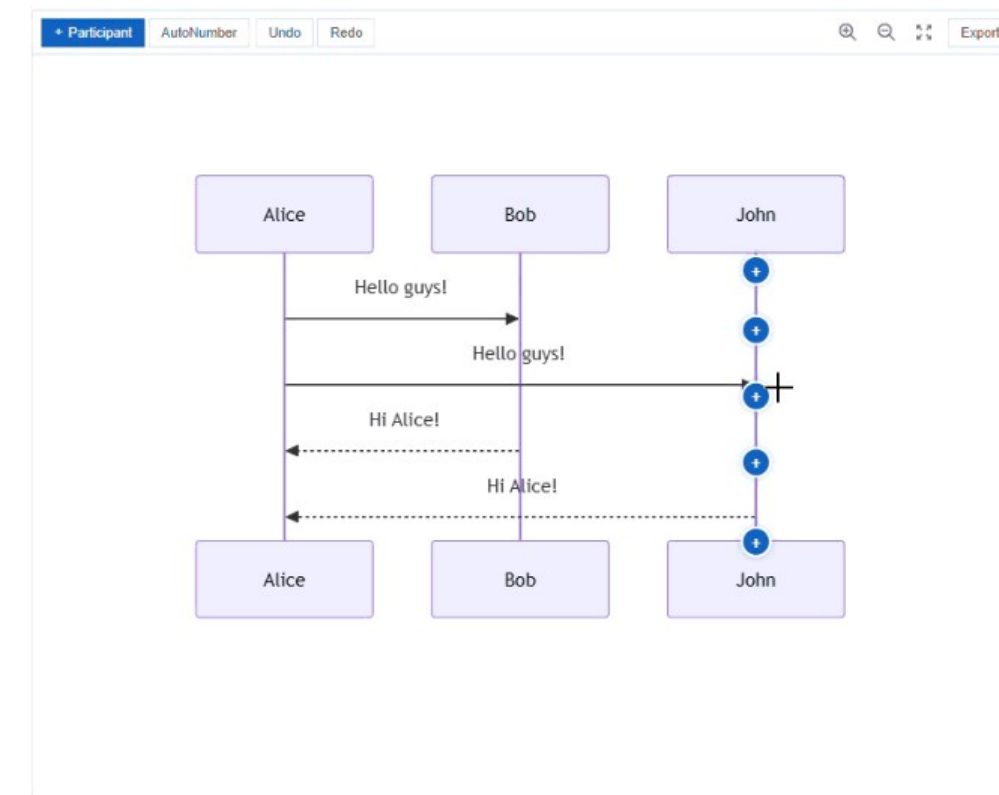
1. Mermaid PlayGround 보다 나은 부분

시퀀스 다이어그램 - par / opt 내부 분기 기능 추가



Block 문법인 par / opt 내부 and / else 분기 문법이 있음
⇒ 하지만 Mermaid PlayGround는 GUI에서 내부 분기 추가 불가능

시퀀스 statements 구조를 기준으로 선택된 메시지 구간이
어떤 block 안에 있는지 계산한 뒤,
par 내부엔 and, opt 내부엔 else 같은 분기 statement를
적절한 위치에 삽입하도록 구현.



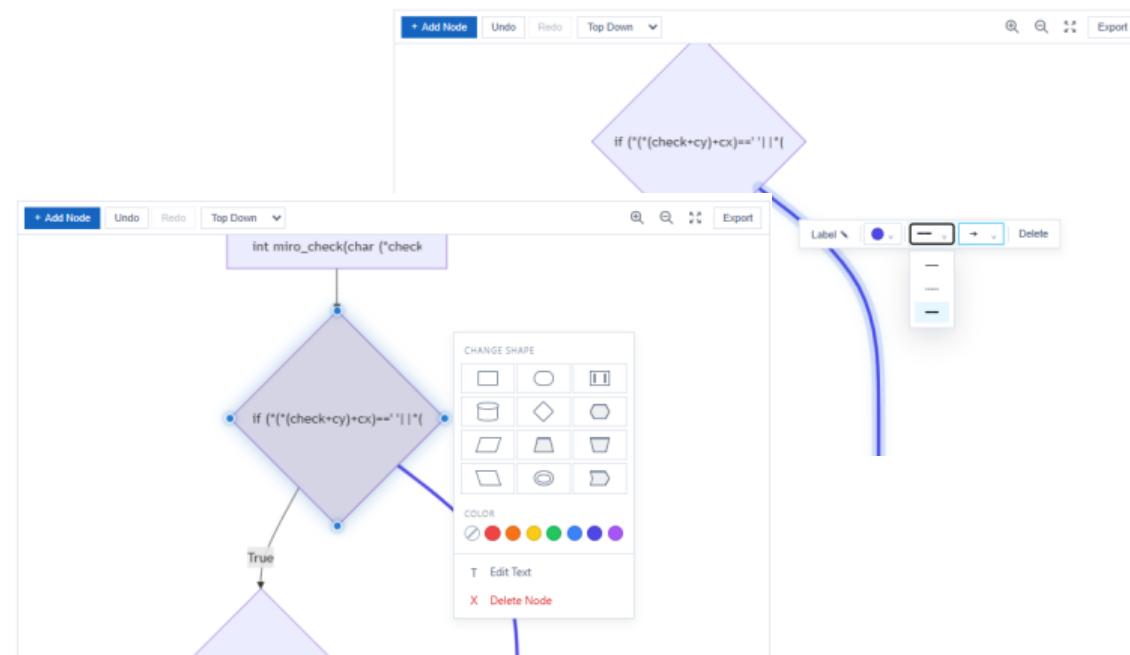
par / opt Block 내부 and / else 분기를 편리하게
추가할 수 있게 개선

02 SureFlow - GUI Editor 개발

SURESOFT

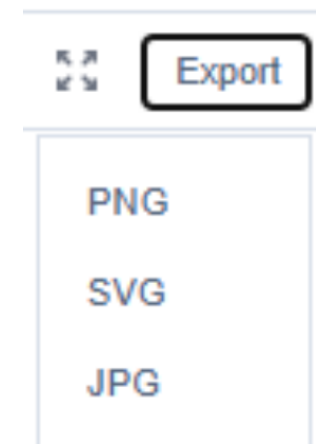
기능 고도화 내용

2. 기능 고도화 부분



플로우차트 - Node 및 Edge 스타일 선택 기능 구현

플로우차트 - 지원 문법 대폭 확대 + 미지원 문법 보존



Export 기능 구현

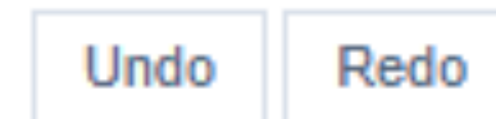
- PNG / SVG / JPG 택 1 가능



zoom / pan / fit view 기능 구현

SVG 접근용 공개 메서드 추가

- exportSvg()
- exportPng()
- exportJpg()
- copySvg()



Redo / Undo 기능 구현

- model 스냅샷 기반

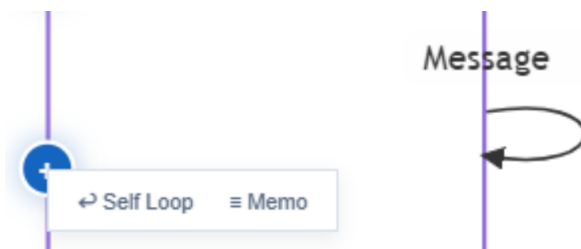
02 SureFlow - GUI Editor 개발

SURESOFT

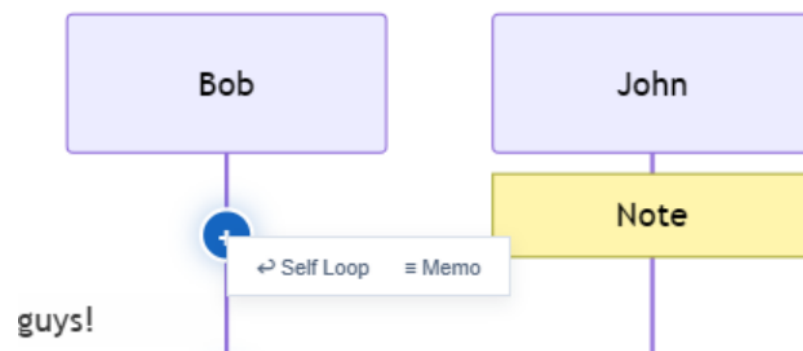
기능 고도화 내용

2. 기능 고도화 부분

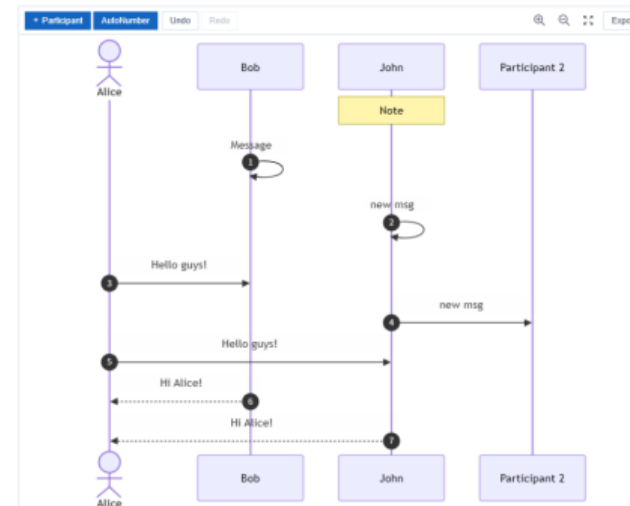
시퀀스 다이어그램 에디팅 기능 구현



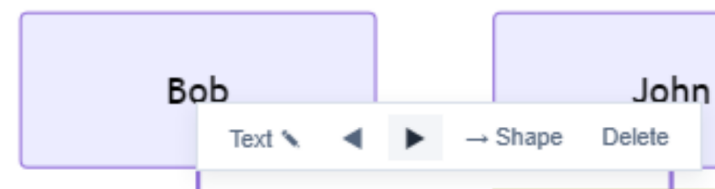
self loop edge 추가 기능 구현



시퀀스 note over 메모 추가 기능 구현



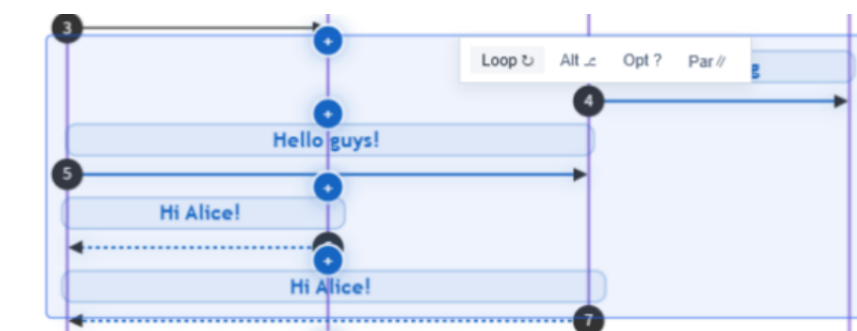
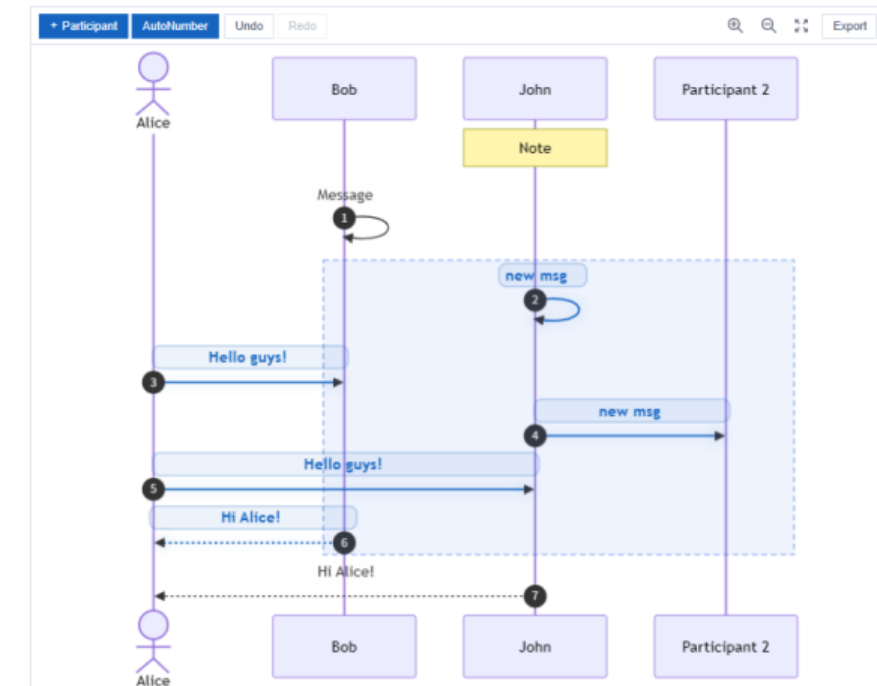
시퀀스 autonumber 지원 추가
- 순서 넘버링 기능



시퀀스 participant 좌우 swap 기능 추가



시퀀스 actor 지원 추가



드래그 및 다중 선택 기능

⇒ 이를 기반으로 block 추가 인터랙션 기능 구현

기능 고도화 내용

3. 사용성 고도화 부분

사용성 개선 - 시퀀스 다이어그램

- 시퀀스 메시지 추가 버튼 위치 수정
- 시퀀스에서 라인이 하나도 없을 때도 첫 라인을 추가할 수 있게 수정
- 시퀀스 participant 사용성 개선
- 시퀀스 툴바 정리
- 시퀀스 라인 편집 사용성 개선
- 시퀀스 메모 편집 UX 개선
- 시퀀스 블록 편집 매핑 수정
- 시퀀스 note 삭제 동기화 문제 수정
- 시퀀스 빈 block 삭제 시 레이아웃 깨짐 버그 수정
- 시퀀스 메모 빈 텍스트 입력 시 자동 삭제 처리
- 시퀀스 block 빈 텍스트 입력 시 자동 삭제 처리
- 시퀀스 nested block에서 title 매핑 어긋남 수정
- 시퀀스 메시지 삽입 핸들 위치 조정
- 시퀀스 드래거 `auto-pan` 기능 추가

사용성 개선 - 플로우차트

- 플로우차트 노드 수정 사용성 개선
- 노드 추가 기능 사용성 개선
- 노드/엣지 편집 UX 개선
- 이름 없는 엣지 클릭 불가 버그 수정
- 엣지 수정 기능 고도화
- 엣지 연결 시 시점 초기화 동작 제거로 사용성 개선
- 엣지 헤드 옵션 UI 개선
- `Change Shape` 안 되던 버그 수정
- `Change Shape` 메뉴 축소 및 정리
- Hexagon 노드 파싱 순서 오류 수정
- Mermaid 코드 파싱 포맷 수정
- Mermaid.js 동적 ID 발급으로 인한 문제 수정
- 포트 드래거 제스처 `auto-pan` 기능 추가

사용성 개선 - 공통

- 미리보기 인터랙션 전반 개선
- 레이아웃 확장
- `fit view` 개선
- GUI 영역 비율 확장
- 전체 UI를 V-SPICE 테마로 교체
- 전체 색상톤을 V-SPICE 톤에 맞게 조정
- Mermaid 테마를 `default` 로 조정
- 임베딩 CSS 주입 에러 수정
- MermaidPreview 전역 리스너 메모리 누수 수정
- `getSvgElement` 렌더 타이밍 개선

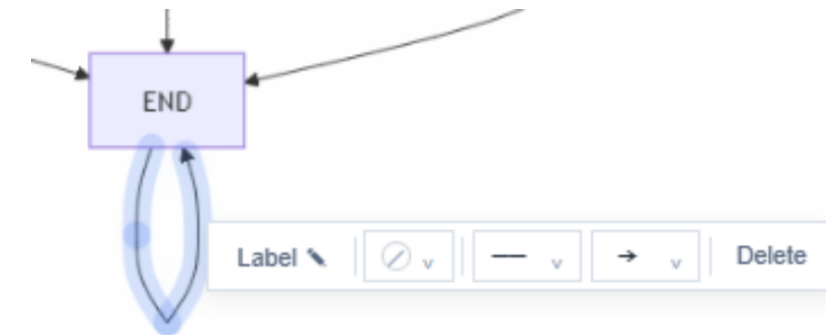
실제 사용을 해보면서 디테일적인 부분 UX 개선

버그 해결 및 안정화

버그 해결

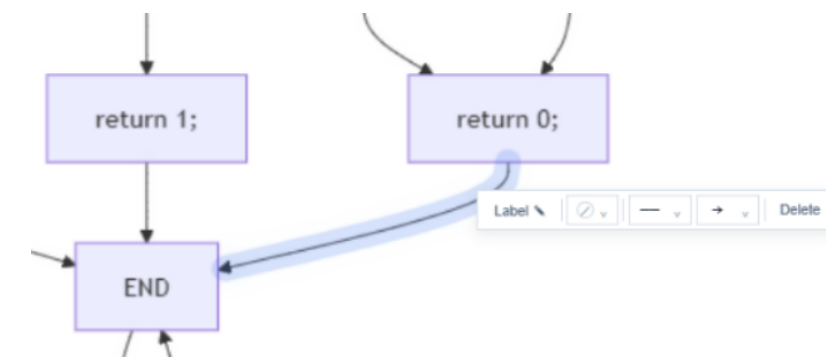
- self loop edge 매핑 버그 해결

- 기: self loop 엣지를 클릭하거나 수정할 때 다른 엣지에 매핑되는 문제
- 승: 원인을 보니 Mermaid가 self loop를 단일 path가 아니라 여러 SVG path 조각으로 렌더
전: 그래서 DOM path 인덱스를 그대로 model edge 인덱스에 대응시키는 방식을 버리고,
self loop는 여러 렌더 슬롯이 같은 edge를 가리키도록 보정
- 결: 결과적으로 self loop 선택, 편집, 스타일 변경이 올바른 엣지에 안정적으로 연결



- 이름 없는 edge 클릭 불가 문제 해결

- 기: 일부 엣지는 label이나 명시적 정보가 없어 클릭 hit target이 제대로 안 잡히는 문제
- 승: 원인은 wrapper 기준으로만 edge를 찾으면 Mermaid 렌더 구조 차이 때문에 실제 stroke path를 놓치는 경우가 있던 점
전: edge wrapper보다 실제 path를 우선 추적하고 ghost hit layer를 덧씌워 클릭 영역을 따로 확보
- 결: 이름 없는 엣지도 일관되게 선택되고 편집할 수 있게 됨



02 SureFlow - GUI Editor 개발

SURESOFT

버그 해결 및 안정화

버그 해결

- Mermaid 동적 ID 발급으로 인한 매핑 오류 해결
 - 기: Mermaid가 렌더할 때 내부적으로 동적 ID를 붙여 노드/엣지 식별이 흔들리는 문제
 - 승: 이 때문에 렌더 결과 DOM과 내부 모델 사이의 연결이 상황에 따라 어긋남
전: 노드/엣지 ID를 추출할 때 sanitize 규칙과 fallback 규칙을 넣고, 모델 기준으로 다시 맞추는 보정 로직을 추가
 - 결: 렌더 결과가 조금 바뀌어도 내부 편집 로직이 덜 깨지는 구조로 개선
- Hexagon 노드 파싱 순서 오류 수정
 - 기: 특정 shape 문법이 다른 패턴보다 먼저/나중에 먹히면서 의도한 도형으로 파싱되지 않는 문제
 - 승: 원인은 shape bracket 패턴이 겹치는데 파서가 먼저 잡아버리는 규칙이 섬세하지 않았던 점
전: 도형 파싱 순서와 bracket 처리 우선순위를 조정
 - 결: shape 변경과 script 파싱 결과가 더 일관되게 맞아떨어지게 됨.

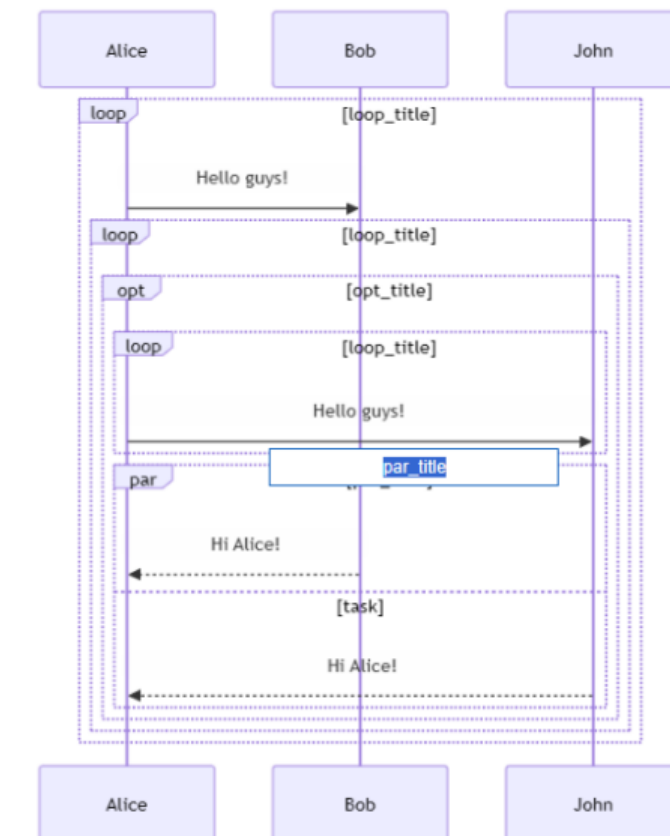
```
var SvgPositionTracker = {  
  // Mermaid가 렌더한 .node 요소에서 논리적인 노드 id를 추출한다.  
  // Mermaid 11은 "[renderPrefix-]flowchart-#{nodeId}-{index}" 형태를 사용한다.  
  extractNodeId: function (nodeEl) {  
    // data-id가 있으면 그 값을 우선 사용한다.  
    var dataId = nodeEl.getAttribute('data-id');  
    if (dataId) return dataId;  
  
    var id = nodeEl.id || '';  
    if (!id) return null;  
  
    // id 안의 "flowchart-" 구간을 찾는다.  
    var marker = 'flowchart-';  
    var idx = id.indexOf(marker);  
    if (idx !== -1) {  
      var after = id.slice(idx + marker.length); // 예: "A-0", "My-Node-3"  
      // 뒤쪽 "-숫자" 인덱스를 제거  
      var m = after.match(/^(?![\s\S]*-)\d+$/);  
      return m ? m[1] : after;  
    }  
  
    // 마지막 fallback: 앞뒤 dash segment를 제거해 추정  
    var parts = id.split('-');  
    if (parts.length >= 3) return parts.slice(1, -1).join('-');  
    if (parts.length === 2) return parts[1];  
    return id;  
  },  
};
```

```
subroutine: ['[', ']', ']',  
cylinder: ['(', ')', ')'],  
rhombus: ['{', '}', '}',  
hexagon: ['{', '}', '}',  
parallelogram: ['/', '/', ']',  
parallelogram_alt: ['\\', '\\', ']',
```


버그 해결 및 안정화

버그 해결

- **nested block title 매핑 어긋남 수정**
 - **기**: 편집 후 selection 꼬임, ghost state, 레이아웃 깨짐이 줄어들었음
 - **승**: 원인은 block 식별을 단순 위치 감으로 처리하면 중첩 구조에서 statement index 대응이 흔들릴 수 있다는 점
 - 전**: block 목록을 구조적으로 계산하고, blockId → statementIndex 기준으로 다시 찾아가도록 보정
 - **결**: 중첩된 alt/par/loop 안에서도 올바른 block만 수정되도록 안정화
- **시퀀스 block/note 편집 동기화 버그 해결**
 - **기**: 시퀀스 블록 제목 편집, note 삭제, 빈 텍스트 입력 후 처리에서 모델과 화면 상태가 어긋나는 문제
 - **승**: 원인은 block/note가 단순 message가 아니라 statements 트리 기준으로 관리돼야 하는데, 편집 후 index/selection 정리가 덜 되어 있었던 점
 - 전**: block/note 갱신과 삭제를 SequenceStatementUtils와 sequenceModelEditing 기준으로 통일하고, 빈 텍스트는 삭제로 처리하도록 정리
 - **결**: 편집 후 selection 꼬임, ghost state, 레이아웃 깨짐이 줄어들었음



```
updateBlockText: function (model, data) {
  if (!model || !data || !data.blockId) return model;
  var nextText = String(data.text || '').trim();
  return finish(model, {
    statements: nextText
    ? SequenceStatementUtils.updateBlockText(model, data.blockId, nextText)
    : SequenceStatementUtils.deleteBlock(model, data.blockId)
  });
},
```

02 SureFlow - GUI Editor 개발

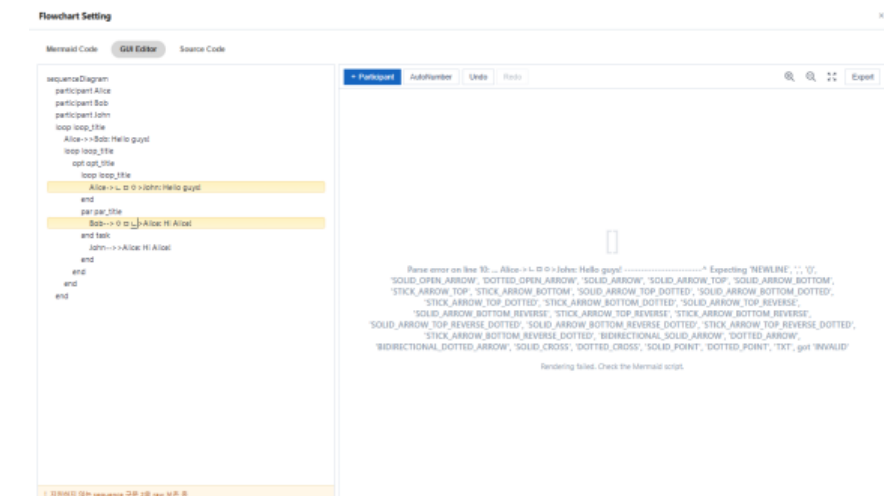
SURESOFT

버그 해결 및 안정화

안정화 (보험 로직)

- 미지원 문법 raw 보존 로직
 - 기: GUI가 아직 완전 지원하지 않는 Mermaid 문법을 만나면 기존엔 손실되거나 의미가 바뀔 위험
 - 승: 특히 parse → model → generate 과정에서 지원 안 되는 줄이 사라지면 사용자가 원본 스크립트를 잃게 됨.
 - 전: 파서에서 미지원 줄을 raw statement로 모델에 그대로 넣고, generator는 그걸 다시 원문 그대로 직렬화
 - 결: 완전 편집은 못 하더라도 unsupported 구문이 안전하게 살아남는 “보험”이 생김
- 파싱 에러/미지원 구간 diagnostics + 하이라이트
 - 기: 사용자는 어디가 GUI 미지원 구간인지, 왜 일부가 편집 대상이 아닌지 알기 어려웠음
 - 승: 단순히 raw로만 남기면 내부적으로는 안전해도 사용자 입장에서는 디버깅이 어려움
 - 전: 파서가 raw 처리된 줄 번호, 원문, 사유를 diagnostics에 기록하고, 에디터는 해당 줄을 하이라이트하도록 연결
 - 결: “왜 이 부분은 GUI가 못 건드리는지”를 즉시 보여줘서 편집 안정성과 설명 가능성이 함께 올라감

```
if (line.indexOf('%') === 0) { model.statements.push({ type: 'raw', raw: line }); co
var sourceInfo = countSourceOccurrence(model, line);
if (line === 'autonumber') { model.autonumber = true; continue; }
if (parseParticipantLine(line, model)) continue;
if (parseMessageLine(line, model)) continue;
if (parseActivationLine(line, model)) continue;
if (parseNoteLine(line, model)) continue;
if (parseControlline(line, model, i + 1, sourceInfo)) continue;
pushRawStatement(model, line, '', i + 1, sourceInfo);
}
```



02 SureFlow - GUI Editor 개발

SURESOFT

버그 해결 및 안정화

안정화 (보험 로직)

- parser fallback / 보험 로직 추가

- 기: Mermaid 문법은 패턴이 다양하고, 예상 못 한 입력이 들어오면 전체 파싱이 깨질 수 있음
- 승: 일부 라인만 해석 실패해도 전체 모델 생성이 무너지면 편집기가 너무 취약
전: 파싱 가능한 건 최대한 모델로 변환하고, 애매한 줄은 raw로 넘기는 fallback 경로를 넣어
전체 파서를 더 관대하게 만들
- 결: 일부 문법을 몰라도 전체 에디터가 죽지 않고 최대한 복구 가능한 방향으로 동작

- parser/generator 왕복 안정성 강화

- 기: GUI 편집기는 결국 script → model → script 왕복이 안정적이어함.
- 승: 여기서 정보가 빠지거나 순서가 틀어지면 한 번만 수정해도 원본 스크립트 품질이 무너짐
전: sequence는 messages뿐 아니라 statements를 별도로 관리하고, flowchart/sequence
모두 generator가 raw/structured statement를 함께 직렬화하도록 정리
- 결: round-trip 이후에도 원문의 구조와 의미를 더 잘 보존하는 방향으로 안정화

```
var statement = parseFlowLine(line, model);
if (!statement) {
  pushRawTarget(model, line, i + 1, 'flow-line', sourceInfo);
  pushRawStatement(model, line);
} else {
  model.statements.push(statement);
}
```

```
function parseSequence(script) {
  if (!script || typeof script !== 'string') {
    return {
      type: 'sequenceDiagram',
      explicitParticipants: false,
      participants: [],
      messages: [],
      statements: [],
      diagnostics: {
        rawStatementCount: 0,
        orphanEndCount: 0,
        unmatchedBlockCount: 0,
        rawLineNumbers: [],
        rawTargets: []
      },
      nodes: [],
      edges: [],
      _sourceTextCounts: {}
    };
  }
}
```


버그 해결 및 안정화

안정화 (보험 로직)

- **getSvgElement / 렌더 타이밍 안정화**
 - **기**: export나 외부 연동에서 SVG DOM을 가져오려 할 때 아직 렌더가 끝나지 않아 null이 떨어질 수 있었음
 - **승**: 이 문제는 export 실패나 외부 preview 연동 오류로 바로 이어짐
전: 실제 canvas DOM 우선, 없으면 svgContent fallback 파싱까지 두는 이중 접근을 넣었음
 - **결**: 렌더 타이밍 차이에도 SVG 접근이 훨씬 안정적으로 동작
- **전역 리스너 메모리 누수 정리**
 - **기**: preview interaction이 커질수록 drag/resize/listener가 남아 있으면 누수나 중복 이벤트가 생길 수 있었음
 - **승**: 이런 문제는 당장은 안 보여도 장시간 사용 시 이상 동작을 일으킴
전: mount/unmount 및 drag 종료 시점에 리스너 해제를 더 엄격하게 정리했음.
 - **결**: 반복 편집이나 임베딩 환경에서 더 안정적으로 동작

```
getSvgElement: function () {  
    var preview = this.$refs.preview;  
    if (!preview) return null;  
    // canvas ref는 v-if="svgContent" 조건이라 렌더 완료 전엔 DOM에 없을 수 있음  
    var canvas = preview.$refs && preview.$refs.canvas;  
    if (canvas) return canvas.querySelector('svg');  
    // fallback: svgContent 문자열에서 파싱 (외부에서 getSvgElement를 직접 호출한 경우)  
    if (preview.svgContent) {  
        var tmp = document.createElement('div');  
        tmp.innerHTML = preview.svgContent;  
        return tmp.querySelector('svg');  
    }  
    return null;  
},
```

```
beforeDestroy: function () {  
    if (this._clickCloseHandler) {  
        document.removeEventListener('click', this._clickCloseHandler);  
        this._clickCloseHandler = null;  
    }  
    if (this._keydownHandler) {  
        document.removeEventListener('keydown', this._keydownHandler);  
        this._keydownHandler = null;  
    }  
    if (this._pointerDownCommitHandler) {  
        document.removeEventListener('mousedown', this._pointerDownCommitHandler, true);  
        this._pointerDownCommitHandler = null;  
    }  
    if (this._suppressClickAfterPanHandler) {  
        document.removeEventListener('click', this._suppressClickAfterPanHandler, true);  
        this._suppressClickAfterPanHandler = null;  
    }  
    if (this._windowResizeHandler) {  
        window.removeEventListener('resize', this._windowResizeHandler);  
        this._windowResizeHandler = null;  
    }  
    if (this._resizeFrame) {  
        cancelAnimationFrame(this._resizeFrame);  
        this._resizeFrame = null;  
    }  
    if (this._panMouseUpHandler) {  
        document.removeEventListener('mouseup', this._panMouseUpHandler);  
        this._panMouseUpHandler = null;  
    }  
},
```

임베드 방식 및 외부 연동 방식

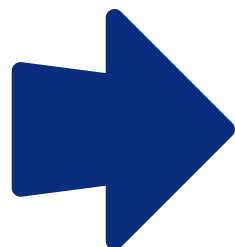
임베드 방식

GUI 에디터 전체를 독립된 컴포넌트 단위로 제공하고,
외부 시스템은 이를 화면에 삽입한 뒤 Mermaid 문자열만 주고받도록 하는 컴포넌트 임베드 방식

임베드 철학

에디터 내부 복잡도는 컴포넌트 안에 캡슐화하고, 외부에는 문자열 입출력 중심의 단순한 계약만 노출한다.

필요할 때만 @svg-rendered, getSvgText(), exportPng() 같은 이벤트나 메서드를
추가로 연결해 외부 연동을 확장할 수 있어서, 기존 시스템을 크게 건드리지 않는 최소 침습 임베드 구조



최소 침습 지향

기존 시스템의 구조와 흐름을 크게 건드리지 않고,
적은 수정으로 기능을 통합할 수 있는 설계 방식

외부 시스템 수정 범위를 최소화하면서 여러 환경에 쉽게 붙이기 위해서!

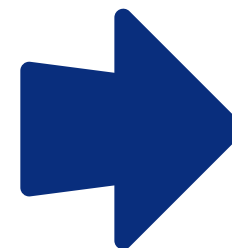
임베드 방식 및 외부 연동 방식

임베드 방식

1. dist/GuiEditor.css + dist/gui-editor.component.js를 로드한 뒤, <mermaid-full-editor>를 Vue 2 전역 컴포넌트로 임베드해서 쓰는 방식
2. 기본 계약은 :value="diagram"로 Mermaid script를 넘기고, @input="diagram = \$event"로 수정된 script를 다시 받는 형태. 즉 외부 앱은 문자열 상태만 관리.

권장 로드 순서

1. Vue 2
2. Mermaid
3. GuiEditor.css
4. gui-editor.component.js
5. 호스트 Vue 코드



최소 침습 지향

기존 시스템의 구조와 흐름을 크게 건드리지 않고, 적은 수정으로 기능을 통합할 수 있는 설계 방식

*** vue와 mermaid 의존성은 호스트가 준비**

```
<link rel="stylesheet" href="/path/to/GuiEditor.css">

<script src="/path/to/vue.min.js"></script>
<script src="/path/to/mermaid.min.js"></script>
<script>
  mermaid.initialize({
    startOnLoad: false,
    theme: 'default',
    securityLevel: 'loose'
  });
</script>
<script src="/path/to/gui-editor.component.js"></script>
```

02 SureFlow - GUI Editor 개발

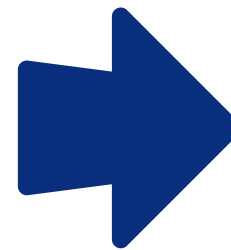
SURESOFT

임베드 방식 및 외부 연동 방식

외부 연동 방식

```
<mermaid-full-editor
  :value="diagram"
  @input="diagram = $event"
></mermaid-full-editor>
```

- props = value
 - 외부에서 현재 Mermaid script를 주입하는 입력값
- events = input
 - GUI 편집 결과로 갱신된 Mermaid script를 외부로 내보내는 용도



외부에서 호출 가능한 메서드들

외부에서 ref를 잡으면 호출 가능한 메서드들

- fitView(), zoomIn(), zoomOut()
- getSvgElement(), getSvgText()
- copySvg()
- exportSvg(), exportPng(), exportJpg()

```
const editor = this.$refs.editor;

editor.fitView();
editor.zoomIn();

const svgEl = editor.getSvgElement();
const svgText = editor.getSvgText();

editor.exportPng();
editor.copySvg();
```

외부에서 메서드 호출해서 사용하는 예시

02 SureFlow - GUI Editor 개발

SURESOFT

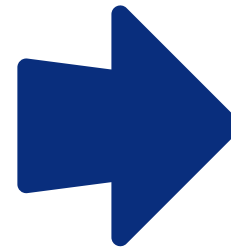
임베드 방식 및 외부 연동 방식

임베드 방식

가장 기본적인 사용 방법은 아래와 같다.

```
<mermaid-full-editor
  :value="diagram"
  @input="diagram = $event"
></mermaid-full-editor>
```

- diagram은 호스트가 관리하는 Mermaid 문자열.
- :value="diagram"은 현재 문자열을 에디터에 내려주는 역할.
- @input="diagram = \$event"는 GUI 편집 결과를 다시 호스트 상태에 반영하는 역할.



임베드 전체 예시

```
<div id="app">
  <div style="height: 700px;">
    <mermaid-full-editor
      :value="diagram"
      @input="diagram = $event"
    ></mermaid-full-editor>
  </div>
</div>

<script>
  new Vue({
    el: '#app',
    data: {
      diagram: [
        'flowchart TD',
        '  A[Start] --> B{Decision}',
        '  B -->|Yes| C[Process A]',
        '  B -->|No| D[Process B]',
        '  C --> E[End]',
        '  D --> E'
      ].join('\n')
    }
  });
</script>
```

화면에 실제로
에디터를 붙여
넣는 자리

Vue 앱을
실제로 실행하는
코드

<mermaid-full-editor> 컴포넌트 임베드

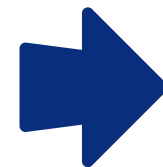
02 SureFlow - GUI Editor 개발

SURESOFT

임베드 방식 및 외부 연동 방식

임베드 방식 - 실제 V-SPICE 내 임베드

```
isGuiView: false,  
  
cfgSaveLoading: false,  
  
//zoom 기능  
cfgZoom: 1.0,  
panX: 0,  
panY: 0,  
isPanning: false,  
panStart: { x: 0, y: 0 },  
},  
watch : {  
  isGuiView : function(newVal) {  
    if (newVal) return;  
    var self = this;  
    this.$nextTick(function() {  
      self.renderMermaid();  
    });  
  },  
  showModal : function(newVal, oldVal) {  
    if(newVal){  
      $('#cfgSettingModal').modal('show');  
    } else {  
      $('#cfgSettingModal').modal('hide');  
      this.unbindPanListeners();  
      this.isGuiView = false;  
      this.isSourceView = false;  
    }  
  }  
}
```



임베드된 GUI 에디터를 기존 모달 UI 안에서 안전하게 열고 닫고,
뷰 전환 후 화면을 다시 맞추기 위한 바깥 제어 코드

즉, **isGuiView** = 모달 안에서 에디터를 켜다 껐다 하고, 나갈 때 화면
과 이벤트 상태를 정리하는 관리 코드

결론적으로,

CfgSettingModal.js와 .jspf 두 파일만 최소 침습으로 임베드 가능

resources\static\lib\js\modal\CfgSettingModal.js

임베드 방식 및 외부 연동 방식

임베드 방식 - 실제 V-SPICE 내 임베드

현재 구조의 가장 큰 특징은 완전한 독립성

- 외부와의 연결은 value, input 두 가지로만 얹게
- 내부 로직, UI, 기능은 모두 Editor 내부에서 처리
- 호스트 앱은 단순히 값을 주고받는 역할만 수행



이 구조 덕분에:

- GUI Editor 내부 로직 수정 가능
- 기능 추가 및 UI 개선 자유롭게 가능
- 호스트 앱 수정 없이도 변경사항 반영 가능

고칠 건 Editor 안에서만 고치고,

외부 인터페이스는 유지하는 방식으로 유지보수와 배포를 빠르고 안전하게 처리할 수 있는 구조

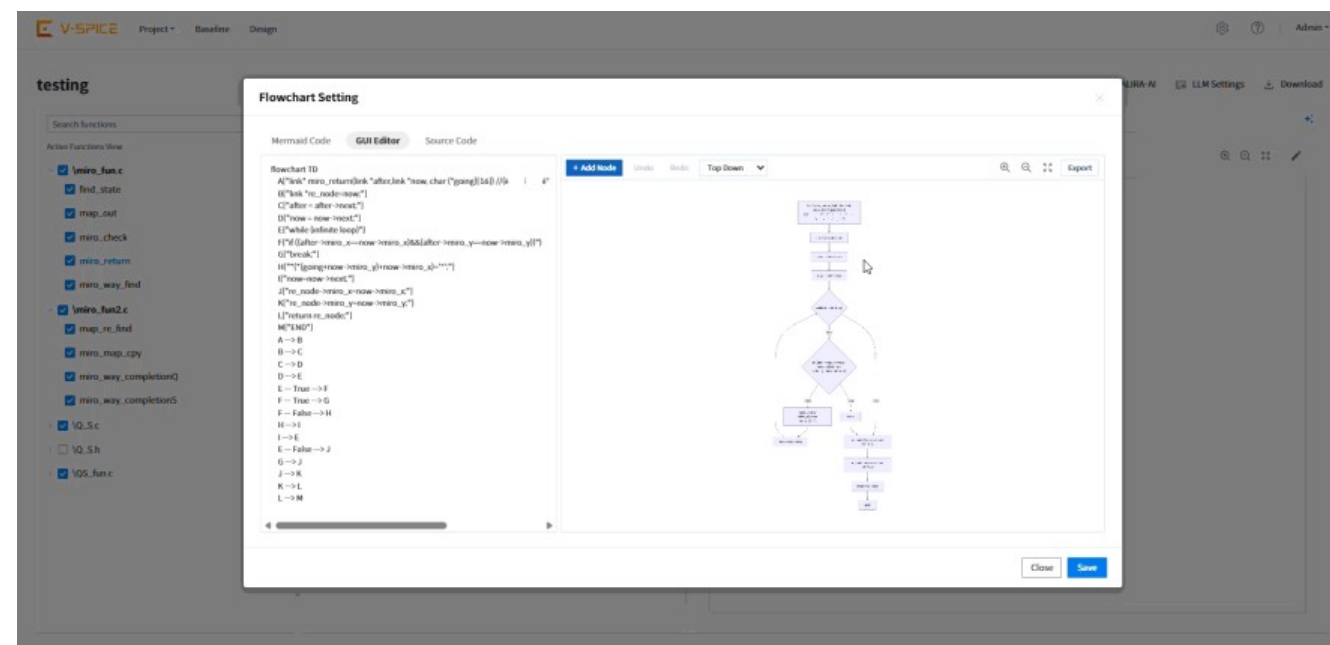
많은 사람들이 쓸수있는 사내 라이브러리로 발전되면 좋겠다.

02 SureFlow - GUI Editor 개발

SURESOFT

Next TODO

앞으로 더 개선할 여지



1. 미지원 Mermaid 문법 지원 확장
2. 외부 연동 이벤트 고도화 - 더 세분화된 이벤트들
3. UX 후속 개선
4. 성능 최적화 (큰 규모의 Diagram에서 Action Handler 재생성 비용 줄이기)
5. State Diagram이나 Flow Diagram과 같은 추가 다이어그램 지원

다음으로 하면 좋을 것을 생각해봤습니다.

**내 작업이 V-SPICE 제품 내에 더 실제 기능으로 남으면 좋겠다!
실제 제품에 남는 방향으로 더욱 더 기여를 하고 싶다.**

1. 백엔드 API 추가 개발 참여

실제 서비스 코드에 기여하는 과정에서 많이 배웠습니다. 가능하다면 하나의 API를 더 맡아서 진행해보고 싶습니다.

2. V-SPICE 내 기존 기능 개선 및 버그 수정

V-SPICE를 실제로 사용해보면서 찾은 버그들을 해결하거나,
전에 제안 드렸던 사용성 개선을 위한 추가 유틸 기능들을 실제로 개발 해보고 싶습니다.

03 Next Step

SURESOFT

다음으로 하면 좋을 것을 생각해봤습니다.

Ver. 1.6.2 기준

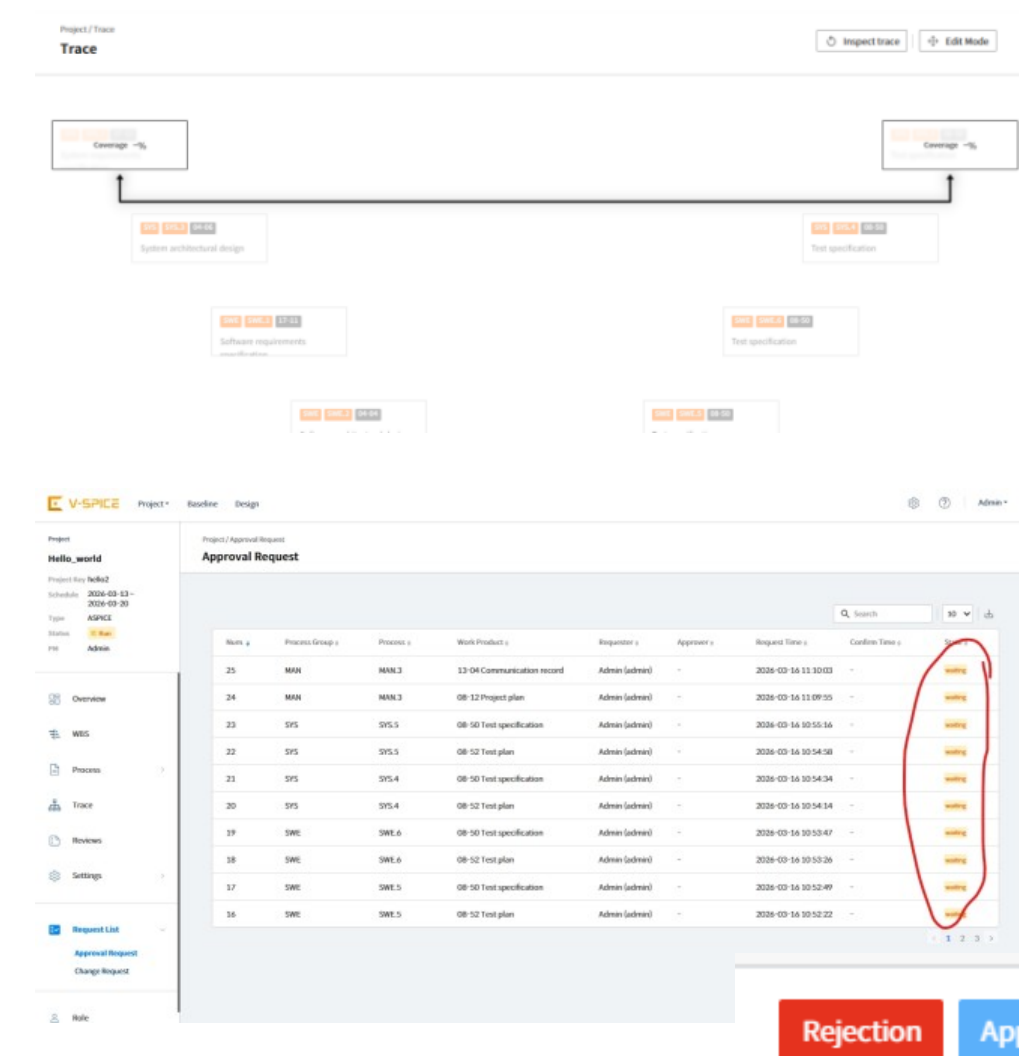
V-SPICE 내 기존 기능 개선 및 버그 수정에 대한 대략적인 구상안

1. Trace 커버리지 Hit Slop 생성 - UX 개선

Trace 커버리지를 보기 위해 클릭 시 선이 너무 얇아서 포인터 겨냥이 힘들.
+ 추가로 선 밑에 작게 Trace Coverage 퍼센테이지(%)를 보여주면 어떨까
연결 안된거는 빨간색 라인으로 표시 ⇒ UX 개선

2. Approval Request Status 미반영 버그

Approve 누르면 반영은 돼서 산출물이 올라가는 가는데
Status는 여전히 Waiting 이라 Change Request를 쓸 수 없음.
STATUS에 대한 Reject/Approve 모두 반영이 안되는 버그.



03 Next Step

SURESOFT

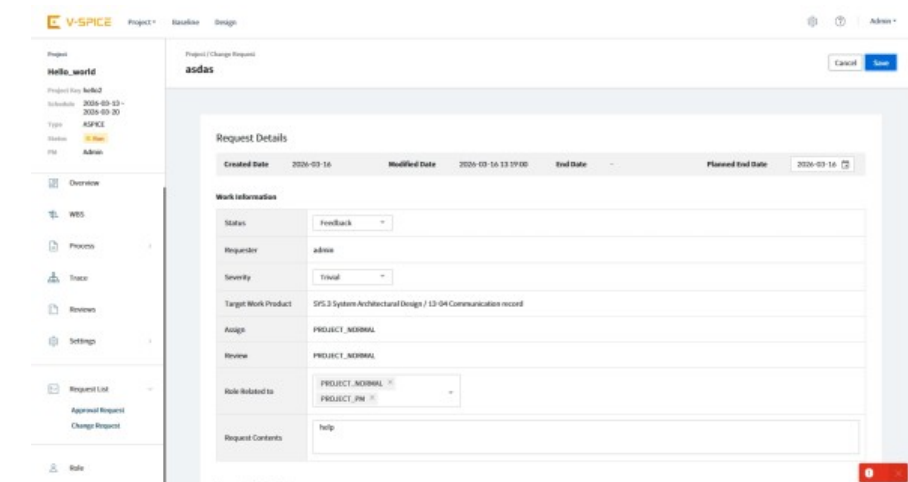
다음으로 하면 좋을 것을 생각해봤습니다.

Ver. 1.6.2 기준

V-SPICE 내 기존 기능 개선 및 버그 수정에 대한 대략적인 구상안

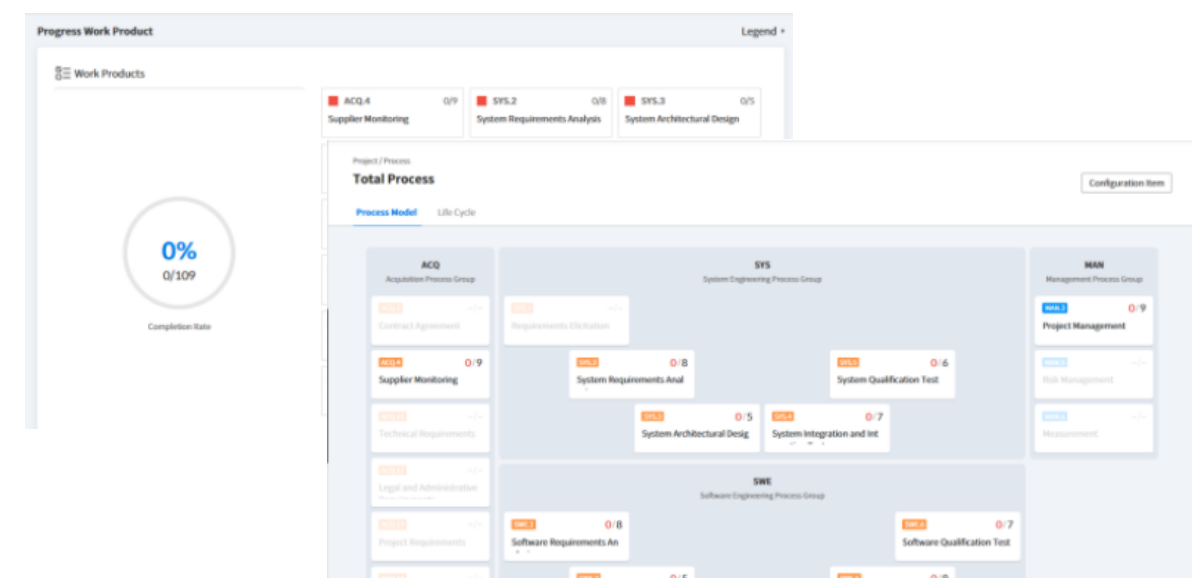
3. Request Details - 간헐적으로 save가 안되는 버그

원인이 확실하진 않은데, 에러 메시지도 빈 체로 Save가 간헐적으로 안됨.
아직 정확한 버그 발동 조건은 찾지 못했음. 10번 중의 1번 꼴로 발생



4. Total Process에도 심사 준비 퍼센테이지(%) 추가

Overview에서 이미 있지만, 많이 사용하는 Total Process에서도 보여주면 더 직관적일거 같음.



03 Next Step

SURESOFT

다음으로 하면 좋을 것을 생각해봤습니다.

Ver. 1.6.2 기준

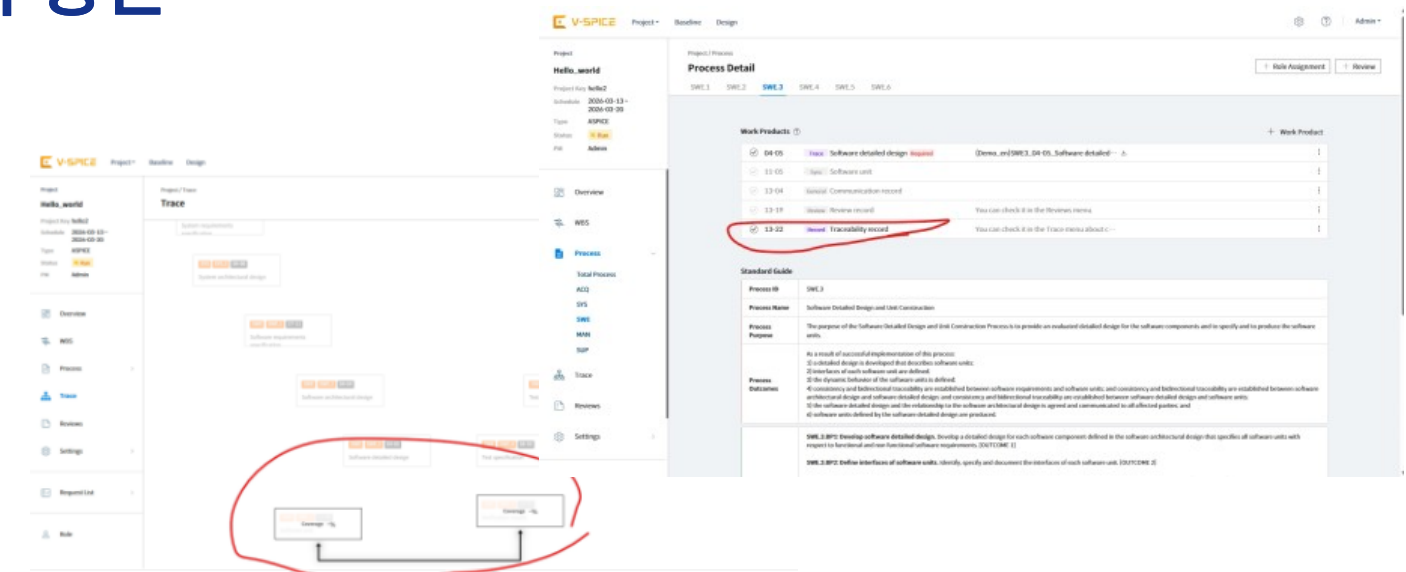
V-SPICE 내 기존 기능 개선 및 버그 수정에 대한 대략적인 구상안

5. SWE 3 & SWE 4 - Traceability Record 버그

두 프로세스 모두 산출물은 알맞게 업로드한 상태.

Trace Coverage ⇒ - % 인 상황.

근데 또 Traceability Record는 만족되었다고 뜨는 버그.

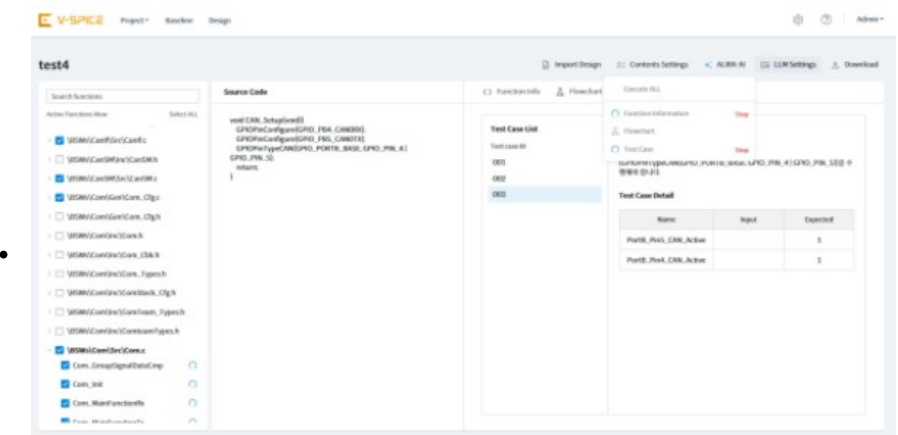


6. Design - ALIRA AI : TC 생성 개선안

프로젝트 내 Function이 너무 많으면 ALIRA 생성 시 오래 걸리다가 중간에 튕기는 문제.

전체 재생성 대신, 나중에 이어서 생성할 수 있게 TC 없는 애들만 생성하는 버튼도 있으면 좋을거 같음.

+ TC 없는 애들이 간략하게 표시되면 어떨까



03 Next Step

SURESOFT

다음으로 하면 좋을 것을 생각해봤습니다.

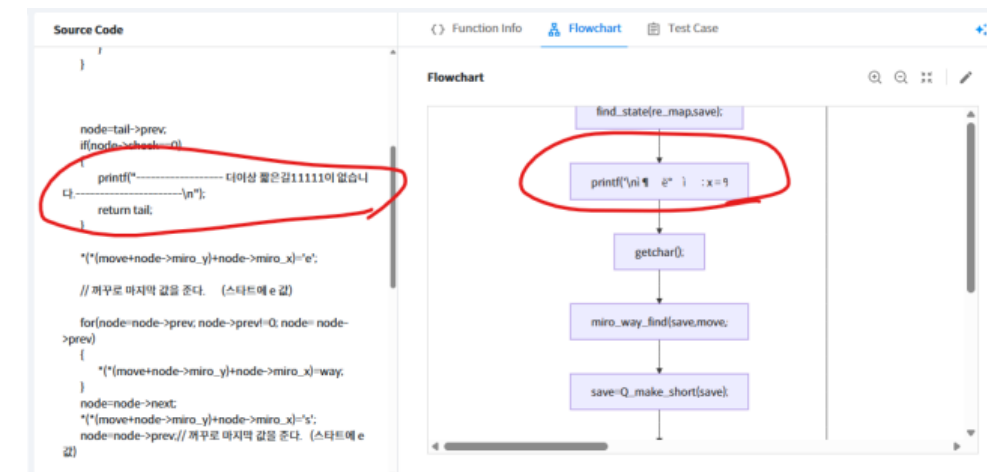
Ver. 1.6.2 기준

V-SPICE 내 기존 기능 개선 및 버그 수정에 대한 대략적인 구상안

7. Mermaid FlowChart - 한글 깨짐 버그

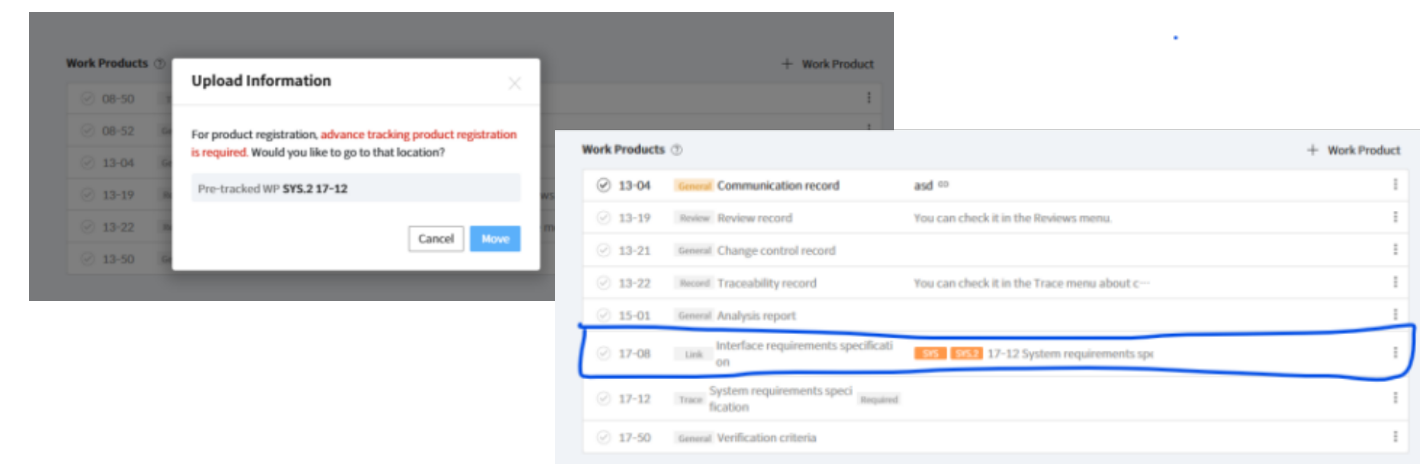
한글이 깨져서 FlowChart 내에 기입되는 문제.

⇒ 인코딩 문제로 보여 쉽게 수정이 가능할 것 같음.



8. Upload Information - MOVE 기능 UX 개선

선행해서 안채워진거 있으면 해당 페이지로 MOVE 시켜주는 좋은 기능!
다만 옮겨진 페이지에서 여러개 중에 타겟이 바로 안보여서
잠깐 반짝이는 이펙트를 넣어서 눈에 띄게해주는것도 좋을 듯함.



03 Next Step

SURESOFT

다음으로 하면 좋을 것을 생각해봤습니다.

Ver. 1.6.2 기준

V-SPICE 내 프로젝트: 코드비머 벤치마킹 개선안

1. 추적성 기반 연관 진입 기능

Review

Change Request

Review Details

asd

Basic Information

Reviewer	Review Date	Process Group	Process	Work Product
Admin	2026-03-16	SWE	SWE.1	13-04

Contents Detail

asd

Attached File

3.12_목_sdd_회의.pdf

Comment 0

ASC | DESC

Please enter a comment.

Overview

WBS

Process

Trace

Reviews

Settings

Request List

Role

Request Details

Created Date	Modified Date
2026-03-16	2026-03-16 13:38:16

Work Information

Status	Completed
Requester	admin
Severity	Major
Target Work	SWE.1 Software Requirements Analysis / 17-11 Software requirements specification
Assign	PROJECT_NORMAL
Review	PROJECT_NORMAL
Role Related to	PROJECT_NORMAL, PROJECT_PM
Request Contents	...

Manage Work Product

Original Work Product	SWE.1.17-11_Software Requirements Specification.xlsx
Changed Work Product	(Demo_en)SWE1.17-11_Software requirements specification.xlsx

V-SPICE에서도 Upstream/Downstream Reference를 클릭해
관련 페이지로 바로 이동 가능하게 제공하면 어떨까



업무 담당자는 해당 Task가 어떤 요구사항과 변경 요청에 근거하는지 즉시 확인 가능

- 업무 맥락 이해 시간 단축
- 개발자/QA가 수정·보완해야 할 대상을 더 직관적으로 파악 가능
- 추적성을 "평가용 정보"가 아니라 "실제 업무 흐름"에 활용 가능

03 Next Step

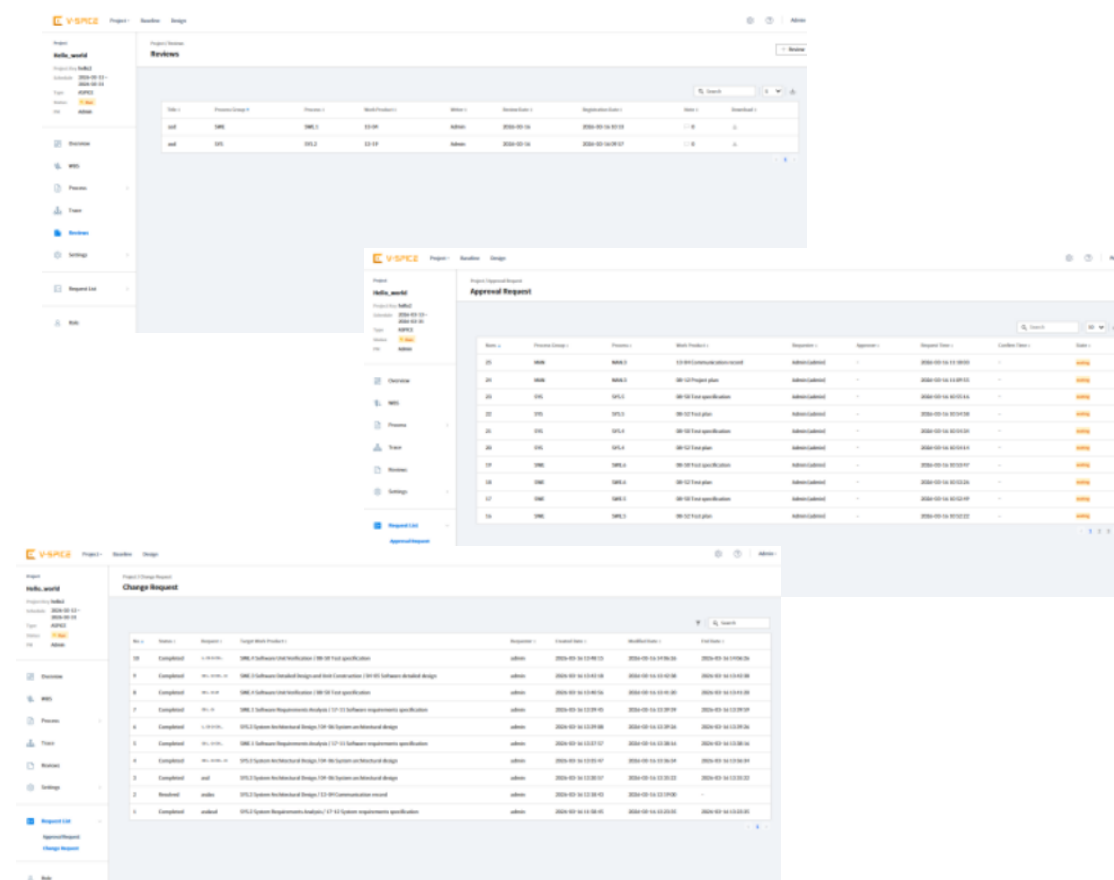
SURESOFT

다음으로 하면 좋을 것을 생각해봤습니다.

Ver. 1.6.2 기준

V-SPICE 내 프로젝트: 코드비머 벤치마킹 개선안

2. Review + Request 통합 허브



현재 3개의 페이지로 나뉘어진 review/requests

V-SPICE에서도 Hub를 제공해주는건 어떨까?
Review + Request를 한곳에 모은 통합 Hub 제공



통합 허브 안에서 Review, Request, Task를 한 흐름에서
통합적으로 관리할 수 있는 구조를 제공

- 추적성과 바로 연결되어, 화면 이동 없이 요청 대상과 관련 산출물을 즉시 파악 가능
- 개발자와 QA가 요청 사항을 더 빠르게 이해하고 처리 가능
- 요청/검토/승인 흐름의 접근성과 업무 속도 향상

03 Next Step

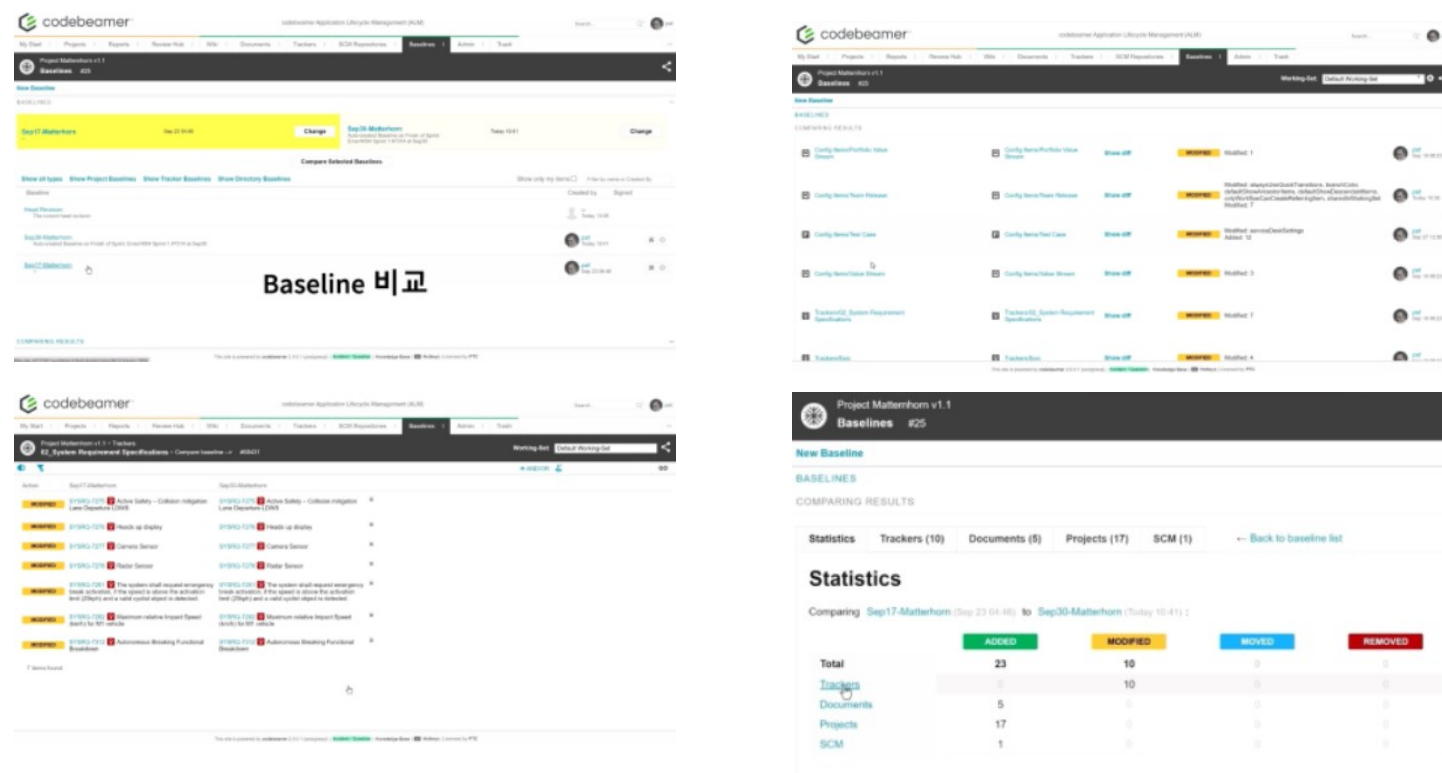
SURESOFT

다음으로 하면 좋을 것을 생각해봤습니다.

Ver. 1.6.2 기준

V-SPICE 내 프로젝트: 코드비머 벤치마킹 개선안

3. Baseline 활용 강화



BaseLine 끼리 비교 후 상세 분석까지도 기능 제공

V-SPICE에서도 BaseLine 기능을 적극 활용해
프로세스 상태를 SnapShot처럼 관리하는건 어떨까?



단계 완료 시 Baseline 생성을 적극 유도하고,
Baseline 간 비교·분석 기능을 강화하여 변경 이력과 형상 상태를 직관적으로
관리할 수 있도록 개선

- 변경 이력 추적과 형상 관리 품질 향상
- 단순 저장이 아니라 시각적 비교와 변경 파악 기능이 함께 제공되면 활용도가 높아짐
- 프로세스 단계별 결과물을 비교·설명하기 쉬움

감사합니다!